Universidade do Minho
Licenciatura em Engenharia Informática

RELATÓRIO

REDES DE COMPUTADORES

TP2 - *PACKET SNIFFER*

Grupo 45

Trabalho realizado por:



Ana Patrícia da Silva Machado
(*a111661*)



Carolina Francisca Vieira Dias
(*a111179*)



Rúben João Mendes Oliveira
(*a93625*)

Maio, 2026

ÍNDICE

1. Arquitetura do Sniffer---- 4
1.1 Visão Geral----4
1.2 Linguagem e Biblioteca---- 4
1.3 Estrutura de Módulos---- 5
1.3.1 sniffer.py----5
2. Protocolos Identificados e Análise---- 7
2.1 Camada de Ligação de Dados: ARP---- 7
Figura 1 - Captura ARP no CORE----8
Figura 2 - Protocolo ARP---- 8
2.2 Camada de Rede: IPv4 / ICMP----8
Figura 3 - Captura ICMP no CORE---- 9
Figura 4 - Captura do comando ping executado no CORE---- 10
Figura 5 - Protocolo ICMP---- 10
2.3 TCP---- 10
Figura 6 - Captura TCP no CORE---- 11
Figura 7 - Protocolo TCP (3-way handshake)---- 12
2.4 UDP----12
Figura 8 - Captura UDP no CORE----13
Figura 9 - Protocolo UDP---- 14
2.5 DHCP----14
Figura 10 - Modo live do sniffer com filtro DHCP---- 15
Figura 11 - Protocolo DHCP (DORA)---- 16
2.6 Outros protocolos---- 16
- DNS---- 16
Figura 12 - Protocolo DNS----18
- QUIC---- 18
Figura 13 - Protocolo QUIC----19
3. Filtros e Parâmetros de Execução---- 19
Figura 14 - Menu Principal----19
3.1 Iniciar captura----20
Figura 15 - Iniciar captura----20
3.2 Ver Tráfego com filtros----20
Figura 16 - Filtrar captura---- 20
3.3 Guardar tráfego em .txt---- 21
Figura 17 - Guardar captura do tráfego---- 21
3.4. Ver estatísticas----22
Figura 18 - Estatísticas---- 22
3.5 Exportar Relatório---- 22
Figura 19 - Guardar o relatório---- 22
3.6 Linha de comandos---- 23
Tabela 1 - Parâmetros de Execução---- 23
4. Mecanismos de Logging---- 24

4.1 Modo Live ---- **24**
Figura 20 - Exemplo de tráfego ---- 25
4.2 Modo Log (ficheiro) ---- **25**
Tabela 2 - Exemplo de Tráfego UDP ---- 25
4.3 Exportação Consolidada de Tráfego (--save) ---- **25**
Figura 21 - Excerto do ficheiro consolidado gerado via --save, com cabeçalho, registo de pacotes e resumo final ---- 26
5. Funcionalidades Opcionais ---- **26**
5.1 Estatísticas Finais de Captura ---- **26**
Figura 22 - Relatório de estatísticas finais apresentado no terminal após Ctrl+C ---- 27
5.2 Cálculo de RTT Médio (ICMP) ---- **27**
Figura 23 - Secção de RTT Médio no relatório final, com amostras recolhidas via ping -4 google.com ---- 28
5.3 Visualização Gráfica (--graph) ---- **28**
Figura 24 - Gráfico gerado pelo sniffer com a distribuição de pacotes e bytes por protocolo ---- 29
5.4 Resolução Automática de Hostnames (DNS Resolver) ---- **29**
Figura 25 - Secção TOP HOSTNAMES RESOLVIDOS no relatório final ---- 30
5.5 Deteção de Anomalias de Segurança ---- **30**
Figura 26 - Alerta de port scan detetado em tempo real no terminal ---- 31
5.6 Captura com Duração Limitada (--duration) ---- **31**
Figura 27 - Captura com duração limitada a 10 segundos via --duration ---- 32
5.7 Identificação Automática de Serviços por Porta ---- **32**
Figura 28 - Identificação automática de serviços conhecidos junto às portas no output em modo live ---- 33
5.8 Suporte a Filtros BPF Nativos ---- **33**
Figura 29 - Filtro BPF nativo a restringir a captura ---- 34
5.9 Menu Interativo de Configuração e Análise ---- **34**
Figura 30 - Menu interativo apresentado quando o sniffer é executado sem argumentos ---- 35
5.10 Estrutura Indexada para Filtragem Pós-Captura ---- **35**
Figura 31 - Aplicação de filtros pós-captura através da hashtable indexada (resultado restrito a TCP/443) ---- 36
5.11 Dashboard Web para Visualização Interativa ---- **36**
Figura 32 - Secção superior do dashboard ---- 37
Figura 33 - Secção inferior do dashboard ---- 38
6. Topologia CORE e Parte A ---- **39**
6.1 Descrição da Topologia ---- **39**
Figura 34 - Topologia do programa core usada para testes ---- 39
6.2 Como a Topologia Permite Identificar os Protocolos ---- **40**
6.3 Capturas Obtidas e Testes ---- **40**
Figura 35 - Resultado da captura de tráfego pelo CORE. ---- 40
6.3.1 Testes ---- **41**
Figura 36 - Resultado dos teste na rede Core ---- 41
7. Testes e Validação ---- **42**
7.1 Testes em Contexto Real ---- **42**
Figura 37 - Relatório de captura ---- 42
7.2 Testes de Filtros ---- **43**

Figura 38 - O uso de filtro TCP após o fim da captura.---------- 43
Figura 39 - Captura Wireshark do protocolo TCP---------- 43
7.3 Testes de Logging---------- 44
Figura 40 - O ficheiro log esperado---------- 44
8. Análise na Interface Real (Parte B)---------- 44
8.1 Configuração e Setup---------- 44
Figura 41 - Comando de execução do sniffer na interface real wlp2s0---------- 45
8.2 Protocolos Observados---------- 45
Figura 42 - Captura de pacotes ICMP---------- 45
Figura 43 - Captura de pacotes ARP---------- 46
Figura 44 - Captura de pacotes TCP com identificação de flags em ligações HTTPS----------46
Figura 45 - Captura de tráfego UDP com queries DNS para o servidor local---------- 46
8.3 Protocolos Extra (não presentes no CORE)---------- 47
Figura 46 - Tráfego UDP/443 identificado como protocolo QUIC, gerado por serviços Google- 47
8.4 Obstáculos Encontrados----------47
9. Limitações da Aplicação---------- 48
10. Conclusão---------- 49

1. Arquitetura do Sniffer

Esta secção descreve a organização interna do sniffer, os seus módulos principais e as decisões de design tomadas durante o desenvolvimento.

1.1 Visão Geral

O funcionamento do sniffer baseia-se num pipeline de processamento de pacotes em tempo real. O sistema opera através de quatro etapas fundamentais que garantem a integridade e a utilidade dos dados capturados:

1. **Ingestão de Dados (Captura):** A aplicação estabelece uma ligação com a interface de rede (física ou emulada), operando em modo promíscuo. Isto permite que a placa de rede capture todos os frames que circulam no segmento, independentemente do destino;

2. **Processamento e Extração (Parsing):** Cada *raw packet* (pacote bruto) é desconstruído camada a camada, especialmente focado nas camadas de L2 a L4. Nesta fase, o motor do sniffer analisa os cabeçalhos para extrair metadados críticos, como: endereços MAC/IP, portas de serviço e indicadores de controlo (flags);

3. **Filtragem e Classificação:** Os dados processados passam por um motor de regras que valida se o pacote corresponde aos critérios definidos pelo utilizador (filtros por protocolo, IP ou MAC). O tráfego é então classificado e associado ao comportamento típico do protocolo identificado;

4. **Persistência e Visualização:** Finalmente, a informação tratada é encaminhada para o **Modo Live**, garantindo o output formatado na consola para monitorização imediata, e para o **Modo Log**, que assegura a persistência dos dados em ficheiro para auditoria e análise offline.

1.2 Linguagem e Biblioteca

Para o desenvolvimento do núcleo do sniffer, optámos pela utilização da linguagem **Python** em conjunto com a biblioteca **Scapy**. Esta escolha fundamenta-se na flexibilidade da linguagem e na capacidade da biblioteca em realizar o *parsing* de diversos protocolos das camadas de ligação, rede e transporte de forma eficiente.

A utilização do **Scapy** permite focar o desenvolvimento na lógica de análise e classificação de tráfego, evitando a complexidade da manipulação direta de *sockets* brutos ou a necessidade de gerir manualmente os *offsets* de memória para cada cabeçalho de protocolo, como seria necessário em linguagens de mais baixo nível, como C. Além disso, a biblioteca facilita a análise das diferentes camadas dos pacotes capturados (Ethernet, IP,

ICMP, TCP, UDP), o que é essencial para cumprir os requisitos de identificação detalhada exigidos no projeto.

A escolha do Scapy foi previamente validada com o docente das aulas práticas, dado que a biblioteca não trivializa a tarefa de identificação de protocolos. O parsing das camadas e a interpretação dos campos continuam a ser feitos explicitamente pelo nosso código.

1.3 Estrutura de Módulos

O projeto está organizado em dois ficheiros principais: o script `sniffer.py`, que implementa a totalidade da lógica de captura, processamento, análise e output em Python, e o ficheiro `sniffer_dashboard.html`, que constitui o dashboard web complementar descrito na Secção 5.11. Esta organização em ficheiro único para o núcleo da aplicação simplifica a instalação e execução, não requerendo configuração de estruturas de diretórios adicionais nem importação de módulos locais externos.

1.3.1 `sniffer.py`

A aplicação foi desenvolvida seguindo uma arquitetura funcional, em que as responsabilidades estão distribuídas por quatro blocos principais dentro do script `sniffer.py`:

- **Bloco de Configuração e Interface:** Implementado com recurso à biblioteca `argparse`, gere todos os argumentos de linha de comandos disponíveis: interface (`-i`), protocolo (`-p`), endereço IP (`--ip`), endereço MAC (`--mac`), ficheiro de log (`-l`), modo live (`--live`), expressão BPF (`--bpf`), duração (`--duration`), exportação de tráfego (`--save`), geração de gráfico (`--graph`) e filtros pós-captura. Em alternativa, quando o programa é executado sem argumentos, é apresentado um menu interativo numerado que permite configurar e iniciar a captura sem recorrer à linha de comandos. A captura iniciada a partir do menu corre numa thread independente, gerida pelo módulo `threading` da biblioteca padrão do Python, permitindo a interação contínua com o menu enquanto os pacotes são processados em segundo plano.

- **Motor de Captura:** Implementado através da função `sniff()` da biblioteca Scapy, este bloco é responsável pela monitorização ativa da interface de rede em modo promíscuo. Suporta filtragem nativa BPF através do parâmetro `filter`, delegando a expressão de filtragem diretamente ao kernel do sistema operativo antes de os pacotes chegarem ao processo Python, com ganhos significativos de desempenho em redes de elevado débito. Suporta ainda paragem automática por temporização, implementada através do parâmetro `stop_filter`.

- **Unidade de Processamento e Análise (`process_packet`):** É o núcleo da aplicação. Para cada pacote recebido, realiza o parsing camada a camada, identifica o protocolo (ARP, ICMP, TCP, UDP), extrai os metadados relevantes (MACs, IPs, portas, flags, TTL) e executa as operações de enriquecimento: resolução de *hostnames* via `socket.gethostbyaddr()` com cache interna, mapeamento de portas conhecidas para nomes de serviço, cálculo de RTT por correlação de pares ICMP echo request/reply, e deteção de anomalias de segurança (port scan e ICMP flood). Os pacotes processados são indexados numa hashtable por protocolo, IP de origem, IP de destino, porta de origem e porta de destino, suportando filtragem pós-captura com complexidade próxima de O(1).

- **Sistema de Output:** Responsável por todos os mecanismos de persistência e visualização. O modo live gere a formatação colorida com códigos ANSI na consola em tempo real. O modo log (`-l`) assegura a escrita incremental em ficheiro. A exportação consolidada (`--save`) gera um ficheiro estruturado com cabeçalho, lista de pacotes e resumo estatístico. No final de cada sessão, é apresentado automaticamente um relatório com estatísticas coloridas, distribuição por protocolo, RTT médio, top hostnames resolvidos e alertas de segurança. Opcionalmente, com o argumento `--graph`, é gerado um gráfico de distribuição de pacotes e bytes por protocolo com recurso à biblioteca `matplotlib`.

Fluxo de Operações:

1. **Inicialização:** leitura de argumentos via CLI ou arranque do menu interativo;
2. **Configuração:** validação da interface, modos de saída e expressão BPF;
3. **Escuta:** ativação do modo promíscuo e captura via `sniff()`, com filtro BPF ao nível do kernel se definido;
4. **Análise:** execução do callback `process_packet` para cada frame recebido;
5. **Enriquecimento:** resolução de hostname, mapeamento de porta, cálculo RTT e indexação na hashtable;
6. **Deteção de anomalias:** verificação de port scan e ICMP flood em tempo real;
7. **Saída:** exibição em consola (live) e/ou escrita em ficheiro (log/save);
8. **Ciclo:** retorno ao passo 3 até interrupção ou expiração do tempo definido;
9. **Relatório final:** apresentação de estatísticas, alertas e geração opcional de gráfico.

2. Protocolos Identificados e Análise

Para cada protocolo capturado, descrevem-se os campos do cabeçalho analisados, o método de identificação implementado e os passos característicos da interação (ex: estabelecimento de ligação, troca de mensagens, terminação).

2.1 Camada de Ligação de Dados: ARP

O sniffer identifica e monitoriza frames ARP para mapear endereços IP a endereços físicos (MAC) no segmento de rede local.

- **Método de Identificação:** O protocolo é identificado na camada de ligação através do campo EtherType com o valor hexadecimal 0x0806;
- **Campos Analisados:** O motor de parsing extrai os endereços IP e MAC, tanto do emissor (*Sender*), como do recetor (*Target*), utilizando os campos *psrc*, *hwsrc*, *pdst* e *hwdst*;
- **Interação Característica:** O programa distingue ARP Request ($op = 1$), onde um nó solicita a resolução de um endereço, de ARP Reply ($op = 2$), que representa a resposta direta com o endereço MAC correspondente.

Excerto de Código relevante:

```
if ARP in pkt:
        proto  = "ARP"
        op     = "Request" if pkt[ARP].op == 1 else "Reply"
        src_ip = pkt[ARP].psrc
        dst_ip = pkt[ARP].pdst
        info   = f"{op}: {src_ip} -> {dst_ip}"
```

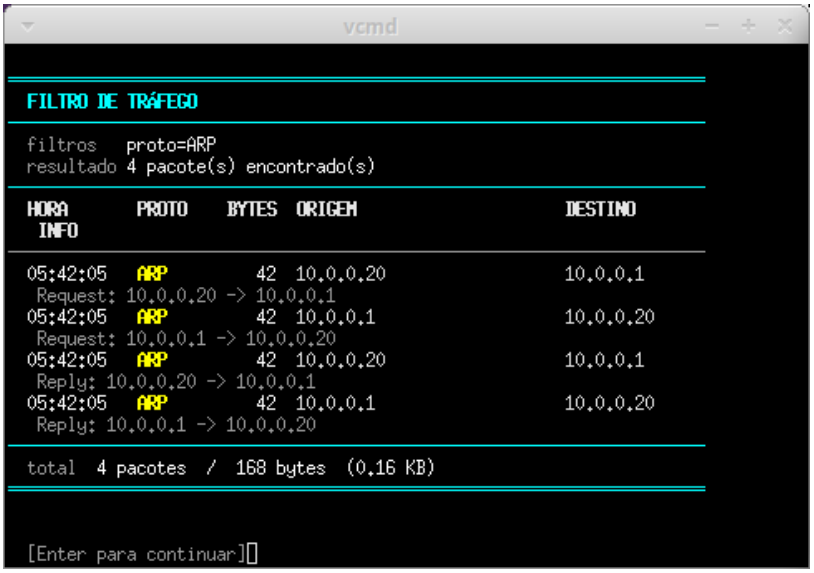


Figura 1 - *Captura ARP no CORE*

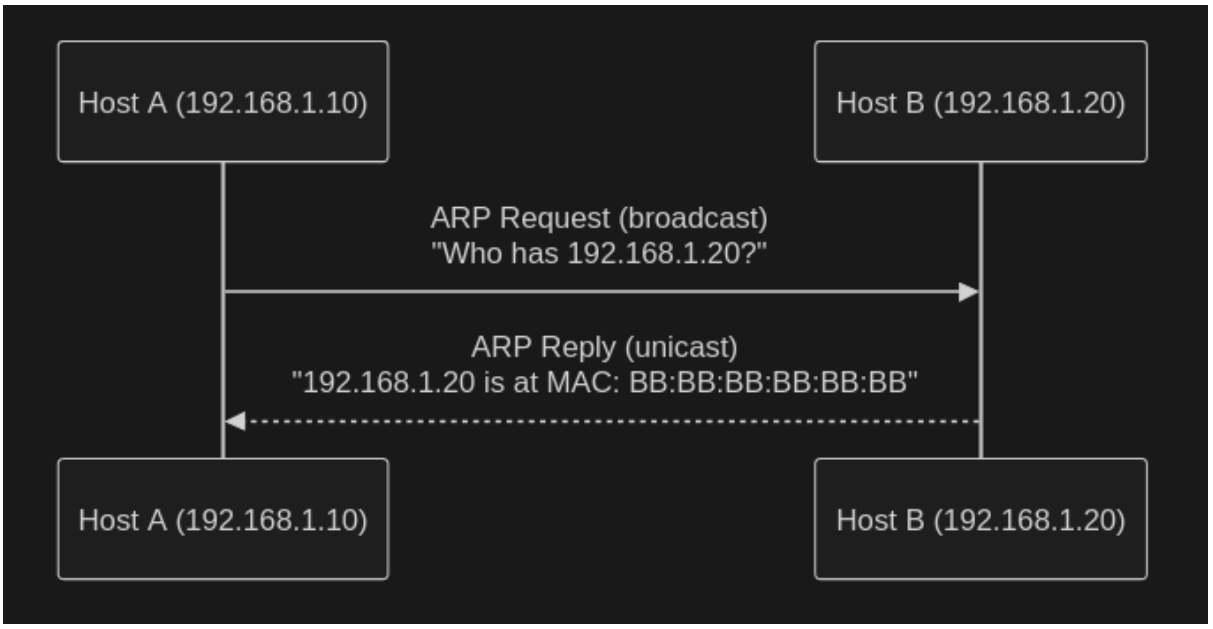


Figura 2 - *Protocolo ARP*

2.2 Camada de Rede: IPv4 / ICMP

Nesta camada, o sniffer foca-se no encaminhamento de datagramas e em mensagens de controlo/diagnóstico.

- **Identificação:** Os pacotes IPv4 são detetados via EtherType 0x0800. Dentro do cabeçalho IP, o protocolo ICMP é identificado quando o campo *protocol* tem o valor 1;

- **Campos Analisados:**

  ➔ IPv4: Endereços de origem/destino (*src/dst*) e o TTL (Time to Live), usado para analisar a validade e o percurso do pacote;
  ➔ ICMP: São extraídos os campos *type* e *code*;

- **Exemplo Prático:** A aplicação identifica mensagens de **Echo Request (Type 8)** e **Echo Reply (Type 0)**, permitindo documentar o fluxo completo de uma interação de conectividade (comando *ping*).

Excerto de Código relevante:

```
elif IP in pkt:
        src_ip = pkt[IP].src
        dst_ip = pkt[IP].dst
        ttl    = pkt[IP].ttl
        if ICMP in pkt:
            proto = "ICMP"
            info  = f"Type {pkt[ICMP].type} (TTL:{ttl})"
            if pkt[ICMP].type == 8:
                icmp_pending[src_ip] = pkt.time
            elif pkt[ICMP].type == 0:
                if dst_ip in icmp_pending:
                    rtt_ms = (pkt.time -
icmp_pending.pop(dst_ip))        * 1000
                    rtt_samples.append(rtt_ms)
                    info = f"Type 0 | RTT: {rtt_ms:.2f} ms
(TTL:{ttl})"
```

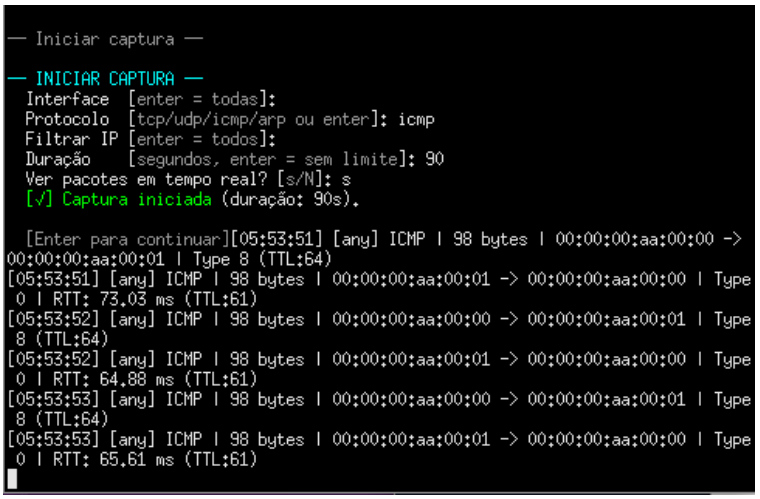


Figura 3 - *Captura ICMP no CORE*

```
root@LOST:/tmp/pycore.40085/LOST.conf# ping -c 3 10.0.5.10
PING 10.0.5.10 (10.0.5.10) 56(84) bytes of data.
64 bytes from 10.0.5.10: icmp_seq=1 ttl=61 time=73.8 ms
64 bytes from 10.0.5.10: icmp_seq=2 ttl=61 time=64.9 ms
64 bytes from 10.0.5.10: icmp_seq=3 ttl=61 time=65.6 ms

--- 10.0.5.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2011ms
rtt min/avg/max/mdev = 64.901/68.123/73.832/4.047 ms
root@LOST:/tmp/pycore.40085/LOST.conf#
```

Figura 4 - *Captura do comando ping executado no CORE*

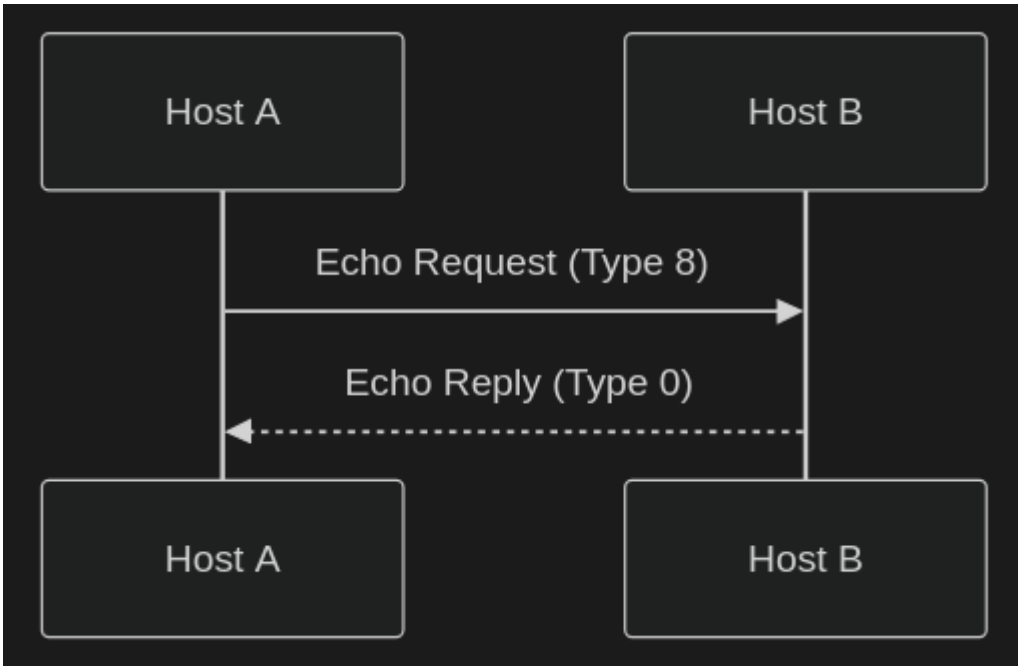


Figura 5 - *Protocolo ICMP*

2.3 TCP

O TCP é um protocolo orientado à ligação, focado na entrega fiável de dados.

- **Identificação:** Detetado dentro do cabeçalho IPv4 quando o campo Protocol apresenta o valor 6;

- **Campos Analisados:** Portas de origem e destino (*sport/dport*), fundamentais para identificar o serviço, e as Flags de controlo;

- **Interação Característica:** A extração das flags (SYN, ACK, FIN, RST) permite ao sniffer documentar o ciclo de vida da ligação:

    1. **Handshake de 3 vias:** Início com SYN, seguido de SYN-ACK e finalizado com ACK;
    2. **Transferência:** Uso da flag PSH (*Push*) para envio de dados;
    3. **Terminação:** Fecho ordenado com as flags FIN e ACK.

Excerto de Código relevante:

```
elif TCP in pkt:
            proto    = "TCP"
            src_port = pkt[TCP].sport
            dst_port = pkt[TCP].dport
            host = resolver_hostname(dst_ip)
            info = f"{src_ip}:{src_port} ->
{host}:{dst_port}{porta_servico(dst_port)} [Flags: {pkt[TCP].flags},
TTL:{ttl}]"
```

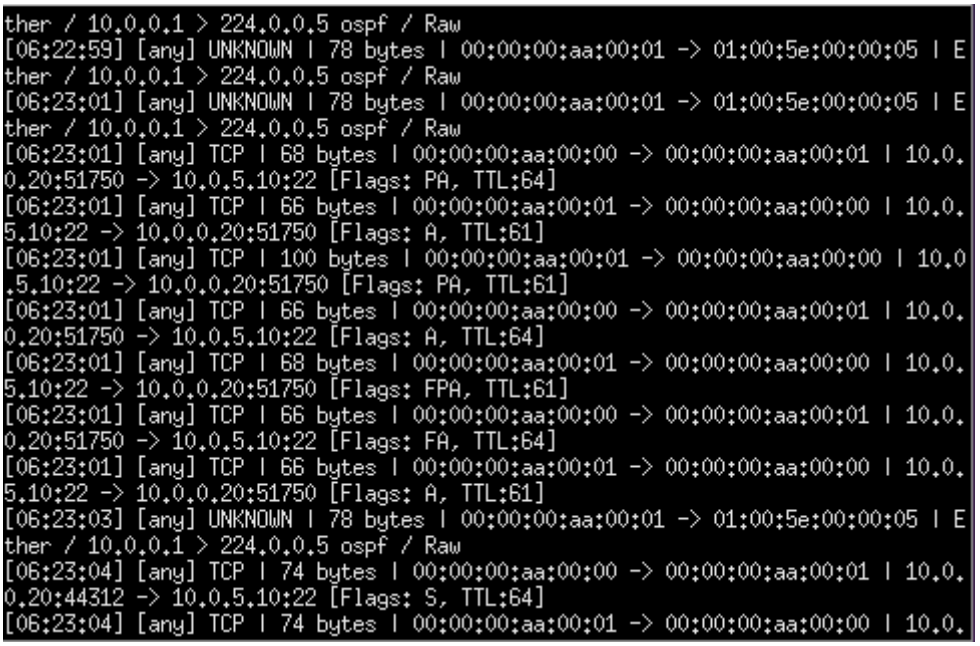


Figura 6 - *Captura TCP no CORE*

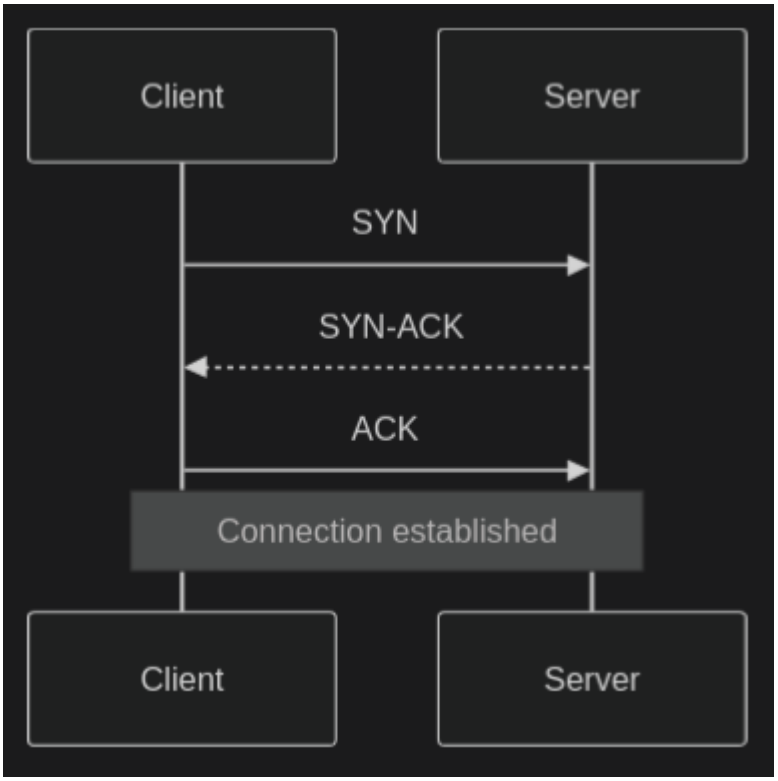


Figura 7 - *Protocolo TCP (3-way handshake)*

2.4 UDP

Ao contrário do TCP, o UDP é um protocolo *connectionless*, priorizando a velocidade em detrimento da confirmação de receção.

- **Identificação:** Identificado no cabeçalho IPv4 através do campo Protocol com o valor 17;

- **Campos Analisados:** Portas de origem e destino, além do comprimento do datagrama para validar a integridade da carga útil;

- **Aplicações Típicas:** O sniffer é capaz de mapear fluxos de serviços comuns, como o DNS (porta 53) ou pedidos de configuração de rede DHCP (portas 67/68), que operam sobre este protocolo.

Excerto de Código relevante:

```
elif UDP in pkt:
            proto    = "UDP"
            src_port = pkt[UDP].sport
            dst_port = pkt[UDP].dport
            host = resolver_hostname(dst_ip)
            info = f"{src_ip}:{src_port} ->
{host}:{dst_port}{porta_servico(dst_port)} [TTL:{ttl}]"
```

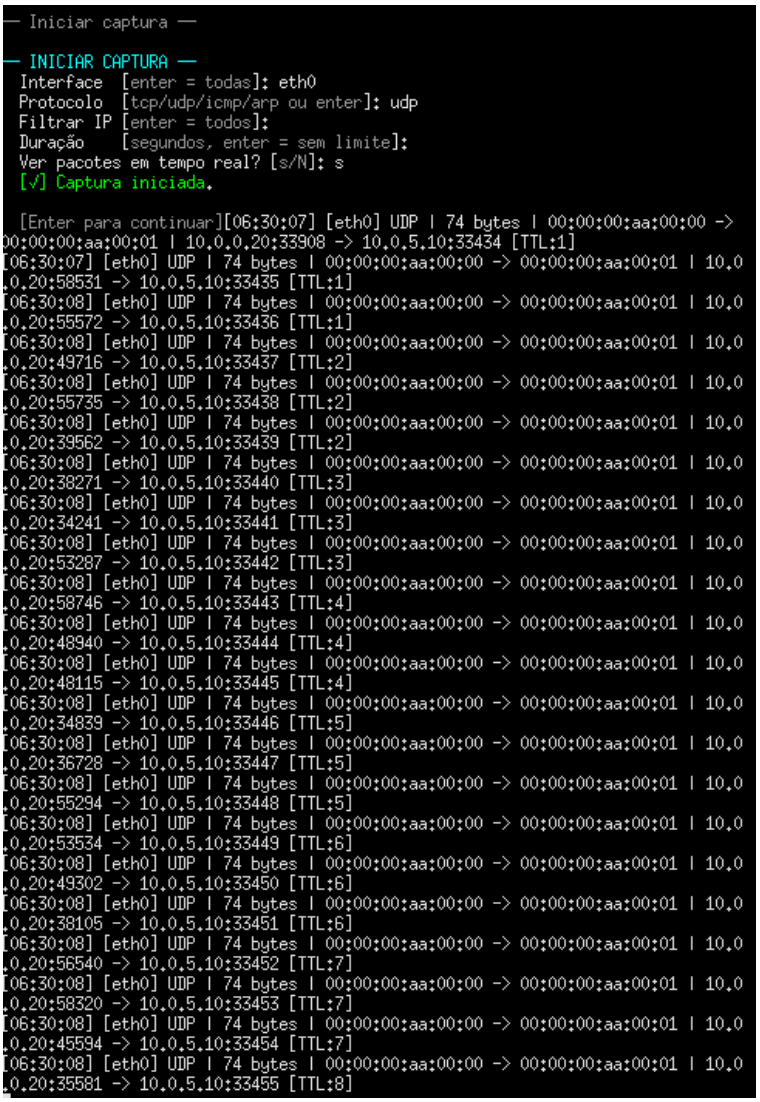


Figura 8 - *Captura UDP no CORE*

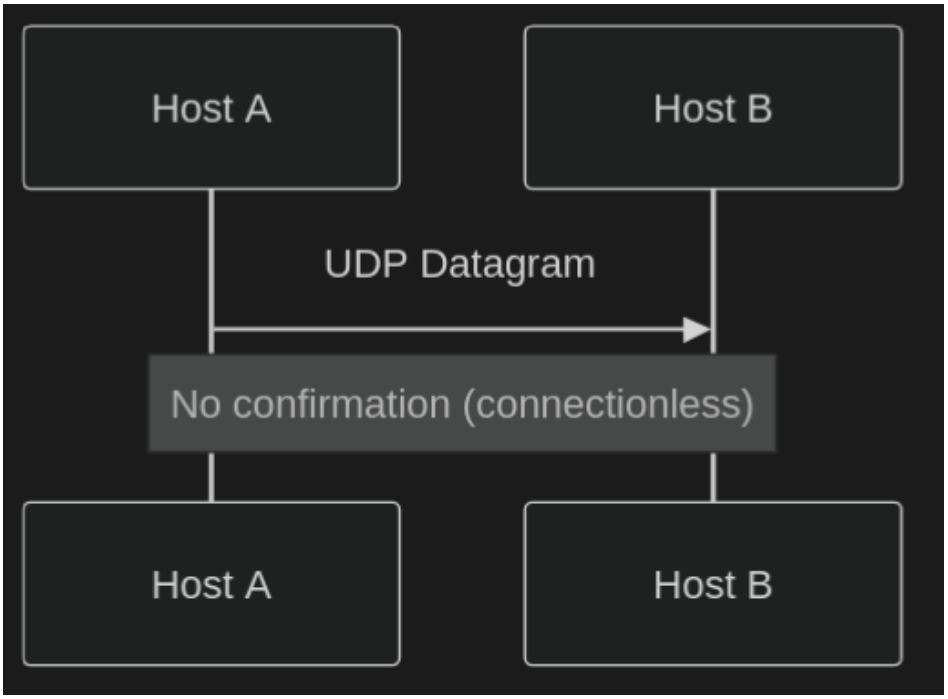


Figura 9 - *Protocolo UDP*

2.5 DHCP

O DHCP (*Dynamic Host Configuration Protocol*) é o protocolo responsável pela atribuição automática de endereços IP e parâmetros de rede aos clientes.

- **Identificação:** Identificado como tráfego UDP com portas 67 (servidor) e 68 (cliente), mapeadas no dicionário **PORT_SERVICES** do sniffer;

- **Campos Analisados:** Portas de origem e destino, que permitem distinguir o sentido da comunicação (cliente→servidor ou servidor→cliente);

- **Interação Característica - Fases DORA:**
  1. **Discover** - o cliente envia um *broadcast* UDP para a porta 67, solicitando a presença de um servidor DHCP na rede;

  2. **Offer** - o servidor responde com uma proposta de endereço IP e configurações de rede (máscara, *gateway*, DNS);

  3. **Request** - o cliente aceita a oferta e envia um pedido formal ao servidor para reservar o endereço proposto;

  4. **Acknowledge** - o servidor confirma a atribuição, concluindo o processo de configuração.

Este protocolo está presente principalmente na interface real (Parte B), uma vez que no ambiente emulado CORE os endereços IP são configurados estaticamente, não havendo necessidade de atribuição dinâmica.

Excerto de Código relevante:

```
PORT_SERVICES = {
    20: "FTP-data",    21: "FTP",        22: "SSH",        23: "Telnet",
    25: "SMTP",        53: "DNS",        67: "DHCP-srv",   68: "DHCP-cli",
    80: "HTTP",        110: "POP3",      123: "NTP",       143: "IMAP",
    443: "HTTPS",      465: "SMTPS",     587: "SMTP-sub",
    993: "IMAPS",      995: "POP3S",     3306: "MySQL",    5432: "PostgreSQL",
    5353: "mDNS",      8080: "HTTP-alt", 8443: "HTTPS-alt"
}


def porta_servico(porta):
    #devolve o nome do servico se a porta for conhecida
    servico = PORT_SERVICES.get(porta)
    return f" ({servico})" if servico else ""
```

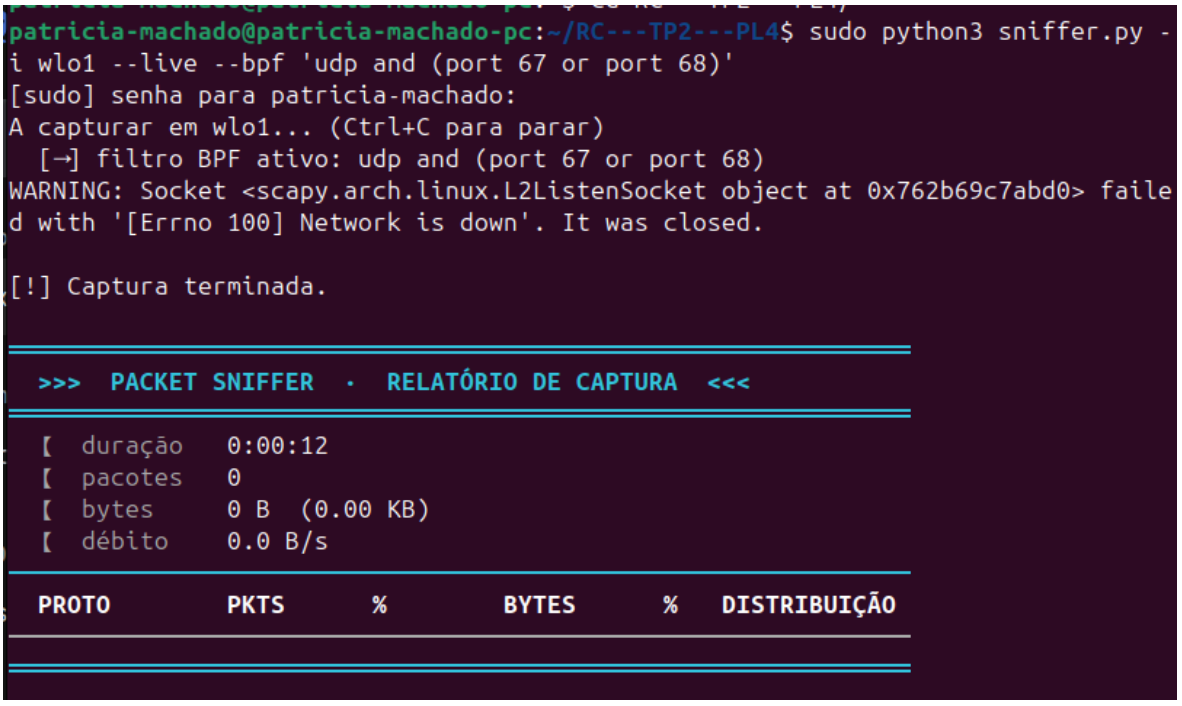


Figura 10 - *Modo live do sniffer com filtro DHCP*

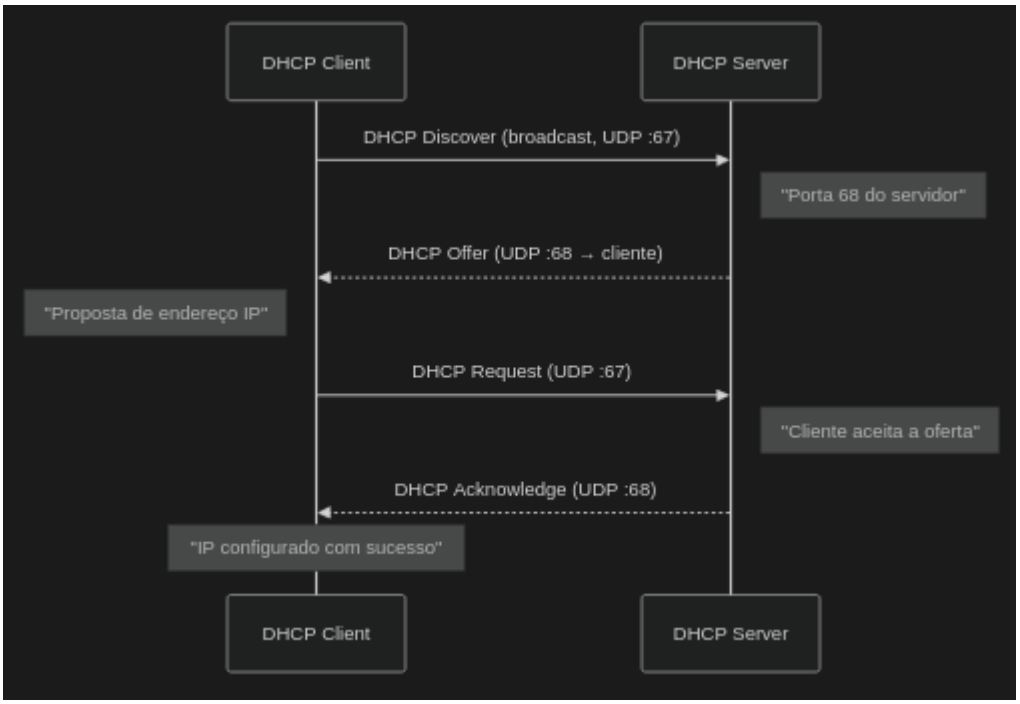


Figura 11 - *Protocolo DHCP (DORA)*

2.6 Outros protocolos

- **DNS**

O DNS (*Domain Name System*) é o protocolo responsável pela resolução de nomes de domínio em endereços IP.

- **Método de Identificação:** Identificado como tráfego UDP com porta de destino 53, mapeada no **PORT_SERVICES** como "DNS";
- **Campos Analisados:** Portas de origem e destino, sendo a porta 53 o indicador do serviço;
- **Aplicação Típica:** Queries geradas automaticamente pelo sistema operativo durante a resolução de nomes de domínio.

Excerto de Código relevante:

```
PORT_SERVICES = {
    20: "FTP-data",    21: "FTP",        22: "SSH",        23: "Telnet",
    25: "SMTP",        53: "DNS",        67: "DHCP-srv",   68: "DHCP-cli",
    80: "HTTP",        110: "POP3",      123: "NTP",       143: "IMAP",
    443: "HTTPS",      465: "SMTPS",     587: "SMTP-sub",
    993: "IMAPS",      995: "POP3S",     3306: "MySQL",    5432: "PostgreSQL",
    5353: "mDNS",      8080: "HTTP-alt", 8443: "HTTPS-alt"
}

def resolver_hostname(ip):
    if ip in dns_cache:
        return dns_cache[ip]
    try:
        hostname = socket.gethostbyaddr(ip)[0]
        if len(hostname) > 35:
            hostname = hostname[:32] + "..."
        dns_cache[ip] = hostname
    except Exception:
        dns_cache[ip] = ip
    return dns_cache[ip]

# Durante o processamento UDP:
elif UDP in pkt:
     proto = "UDP"
     src_port = pkt[UDP].sport
     dst_port = pkt[UDP].dport
     host = resolver_hostname(dst_ip)
     info = f"{src_ip}:{src_port} ->
{host}:{dst_port}{porta_servico(dst_port)} [TTL:{ttl}]"
```

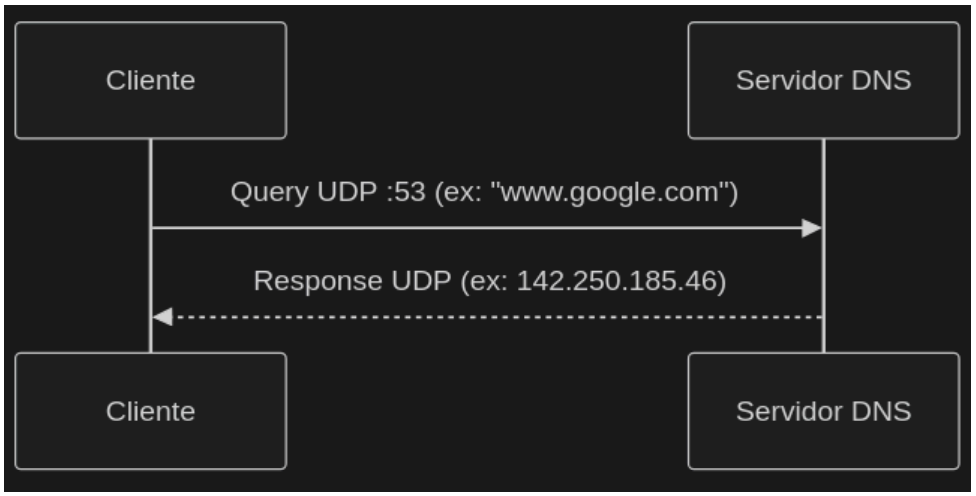


Figura 12 - *Protocolo DNS*

- **QUIC**

O QUIC (*Quick UDP Internet Connections*) é um protocolo de transporte moderno que oferece fiabilidade e segurança sobre UDP.

- **Método de Identificação:** Identificado como tráfego UDP com porta de destino 443, distinguindo-se do HTTPS por usar UDP em vez de TCP;
- **Campos Analisados:** Portas de origem e destino, e endereço IP de destino para identificar a infraestrutura associada;
- **Interação Característica:** Encapsula controlo de fluxo, retransmissão e cifra numa única camada sobre UDP, eliminando o handshake TCP+TLS e reduzindo a latência.

Excerto de Código relevante:

```
PORT_SERVICES = {
    20: "FTP-data",    21: "FTP",        22: "SSH",        23: "Telnet",
    25: "SMTP",        53: "DNS",        67: "DHCP-srv",   68: "DHCP-cli",
    80: "HTTP",        110: "POP3",      123: "NTP",       143: "IMAP",
    443: "HTTPS",      465: "SMTPS",     587: "SMTP-sub",
    993: "IMAPS",      995: "POP3S",     3306: "MySQL",    5432: "PostgreSQL",
    5353: "mDNS",      8080: "HTTP-alt", 8443: "HTTPS-alt"
}
```

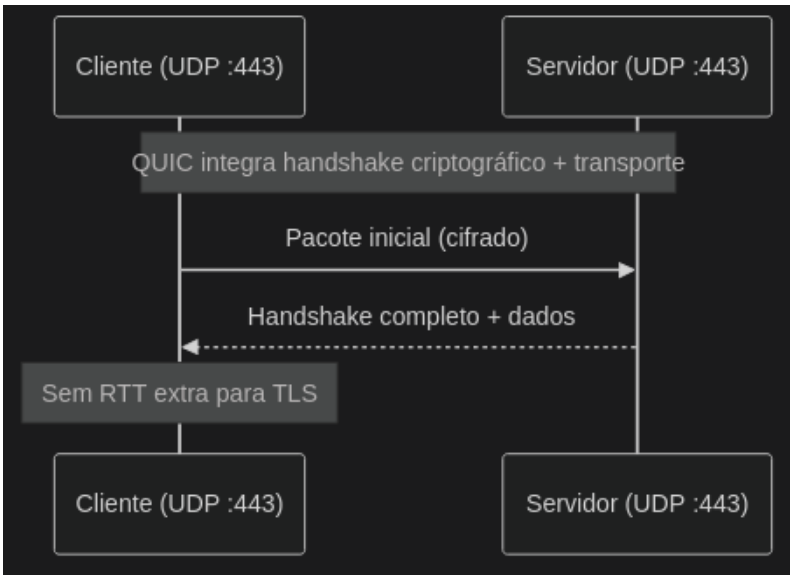


Figura 13 - *Protocolo QUIC*

Este protocolo foi observado na interface real (Parte B), com tráfego gerado por aplicações em segundo plano para endereços Google (**`160.79.104.10`**) e Microsoft (**`52.97.117.2`**). Não é reproduzível no ambiente CORE.

Embora o sniffer apresente a etiqueta (HTTPS) devido ao mapeamento da porta 443, **a análise do payload** (primeiro byte = 0xc0) **permitiria distinguir QUIC de outros tráfegos UDP na mesma porta.**

3. Filtros e Parâmetros de Execução

Ao iniciar o programa com `sudo python3 sniffer.py`, aparece o seguinte menu:

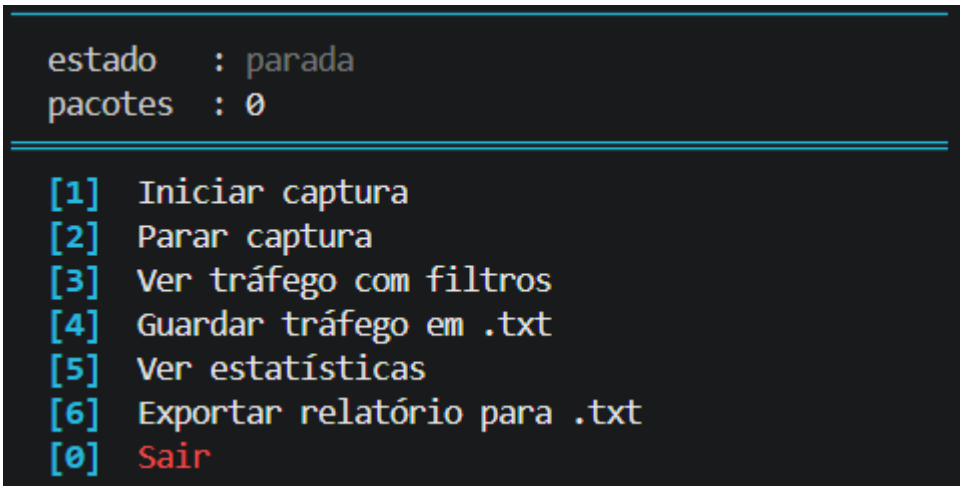


Figura 14 - *Menu Principal*

Ao iniciar o script do packet sniffer, temos o menu principal em que é apresentado se a captura está em curso e o número de pacotes capturados. De seguida temos as opções, que são as seguintes: Iniciar/Parar a captura do tráfego, de seguida temos a consulta do tráfego capturado com o uso de filtros, a quarta opção permite guardar o tráfego num ficheiro txt, já a quinta permite ver várias estatísticas, a sexta opção permite exportar o relatório que é um resumo do que se passou na captura, e por último temos a opção de sair do programas.

3.1 Iniciar captura

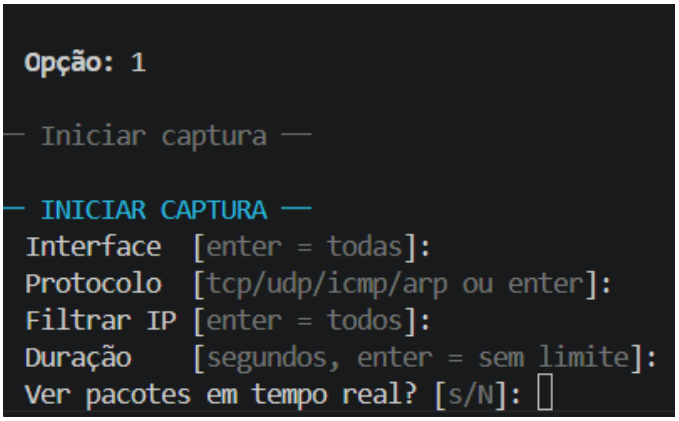


Figura 15 - *Iniciar captura*

Ao iniciar a captura, o utilizador tem a possibilidade de capturar tráfego de várias maneiras como pela interface, protocolo, IP, ou o tempo que esteja a capturar ou mesmo ignorar todos os filtros de captura.

3.2 Ver Tráfego com filtros

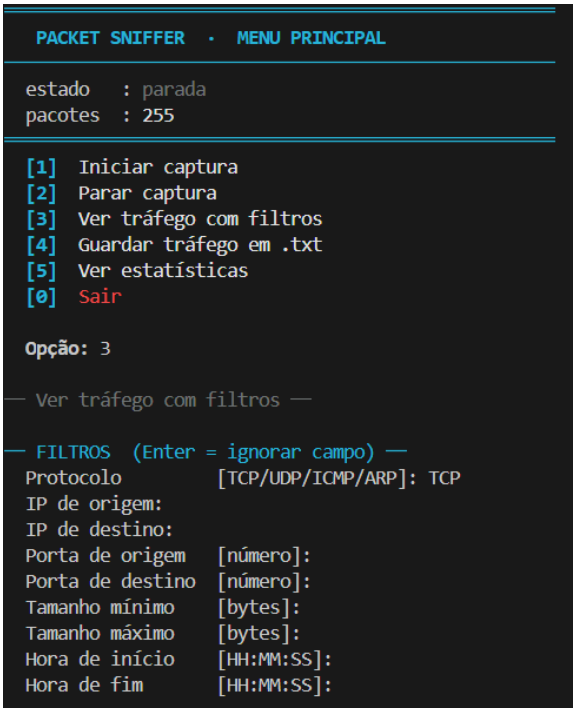


Figura 16 - *Filtrar captura*

Após a captura ser efetuada e parada, o utilizador pode filtrar a captura de várias maneiras, por protocolo, IP origem e destino, o mesmo com a porta que recebe o tráfego, o tamanho dos pacotes, como a hora do fim e do início que foi iniciado e recebido ou simplesmente ignorar por completo.

3.3 Guardar tráfego em .txt

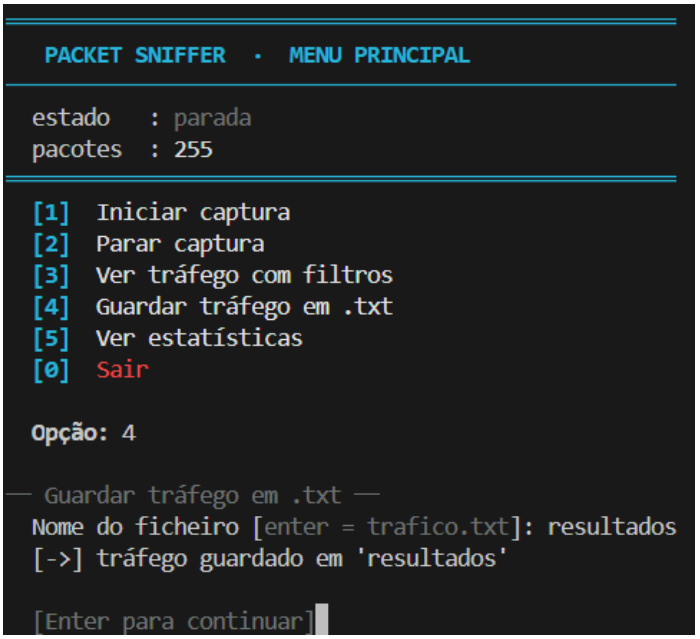


Figura 17 - *Guardar captura do tráfego*

Guardar o tráfego basicamente o utilizador só precisa escrever o nome do ficheiro onde quer que os dados sejam guardados.

3.4. Ver estatísticas

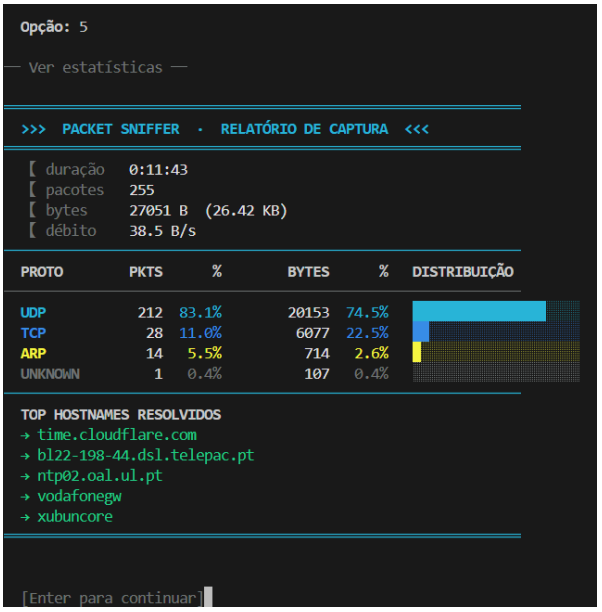


Figura 18 - *Estatísticas*

Na opção estatísticas vemos um mini relatório que mostra a duração da captura, quantidade total dos número de Bytes e o débito usado, logo a seguir temos uma pequena tabela que mostra quais os protocolos que foram capturados, por último temos as lista de host usados.

3.5 Exportar Relatório

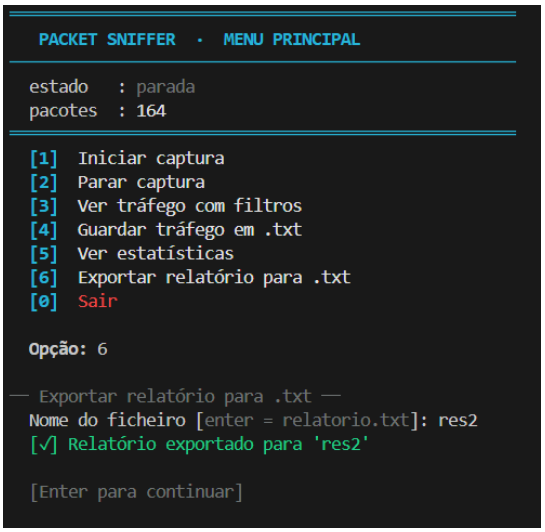


Figura 19 - *Guardar o relatório*

O processo de exportação do relatório é análogo ao de guardar o tráfego: basta indicar o nome do ficheiro de destino.

3.6 Linha de comandos

O sniffer disponibiliza os seguintes parâmetros de execução em linha de comandos:

Tabela 1 *- Parâmetros de Execução*

Parâmetro	Exemplo	Descrição
`--interface / -i`	`-i eth0`	Seleciona a interface de rede a capturar
`--protocol / -p`	`-p tcp`	Filtrar por protocolo (arp, icmp, tcp, udp, dhcp, ...)
`--ip`	`--ip 192.168.1.1`	Filtrar por endereço IP de origem ou destino
`--mac`	`--mac aa:bb:cc:...`	Filtrar por endereço MAC
`--log / -l`	`-l captura.csv`	Ativa modo log e define ficheiro de saída
`--live`	`--live`	Ativa impressão em tempo real na consola
`--graph`	`--graph`	Gera gráfico de barras no final da captura
`--duration`	`--duration 30`	Para a captura automaticamente após N segundos
`--save`	`--save sessao.txt`	Guarda o tráfego capturado num ficheiro .txt no final (default: trafico.txt)
`--fproto`	`--fproto TCP`	Filtra o resultado pós-captura por protocolo
`--fsrc`	`--fsrc 192.168.1.5`	Filtra o resultado pós-captura por IP de origem
`--fdst`	`--fdst 8.8.8.8`	Filtra o resultado pós-captura por IP de destino
`--fsport`	`--fsport 52341`	Filtra o resultado pós-captura por porta de origem
`--fdport`	`--fdport 443`	Filtra o resultado pós-captura por porta de destino
`--fmin`	`--fmin 500`	

		Filtra o resultado pós-captura por tamanho mínimo em bytes
`--fmax`	`--fmax 1500`	Filtra o resultado pós-captura por tamanho máximo em bytes
`--fhi`	`--fhi 10:00:00`	Filtra o resultado pós-captura a partir de uma hora
`--fhf`	`--fhf 10:05:00`	Filtra o resultado pós-captura até uma hora
`--bpf`	`--bpf 'tcp port 443'`	Aplica filtro BPF nativo ao nível do kernel (sintaxe estilo tcpdump/Wireshark)

4. Mecanismos de Logging

O sniffer suporta dois modos de output, podendo ambos estar ativos em simultâneo:

4.1 Modo Live

O modo *Live* permite a monitorização em tempo real do tráfego que atravessa a interface de rede. O *output* foi desenhado para ser intuitivo e informativo, utilizando uma estrutura de colunas fixa que facilita a leitura rápida (*scannable*).

- **Estrutura da Linha de Output:** Cada pacote capturado é processado e impresso numa única linha, seguindo o padrão: [Timestamp] [Interface] Protocolo | Tamanho bytes | MAC src -> MAC dst | Info Detalhada

    1. **Timestamp:** Registo da hora exata da captura (HH:MM:SS), essencial para a correlação temporal de eventos;
    2. **Protocolo:** Identificação da camada de rede ou transporte (ARP, ICMP, TCP, UDP). *Nota: Caso o tráfego detetado não pertença à família IPv4/ARP (ex: tráfego IPv6), o sistema classifica-o como UNKNOWN para sinalizar protocolos fora do âmbito de análise pretendido;*
    3. **Tamanho:** Comprimento total do pacote em bytes;
    4. **Info Detalhada:** Campo dinâmico que varia conforme o protocolo, exibindo metadados críticos como portas de serviço, *flags* de controlo ou endereços IP.

- **Vantagem Técnica:** Esta formatação permite ao administrador de rede identificar rapidamente padrões de tráfego, comportamentos anómalos ou picos de carga sem a necessidade de recorrer a ferramentas externas de análise (*off-line*).

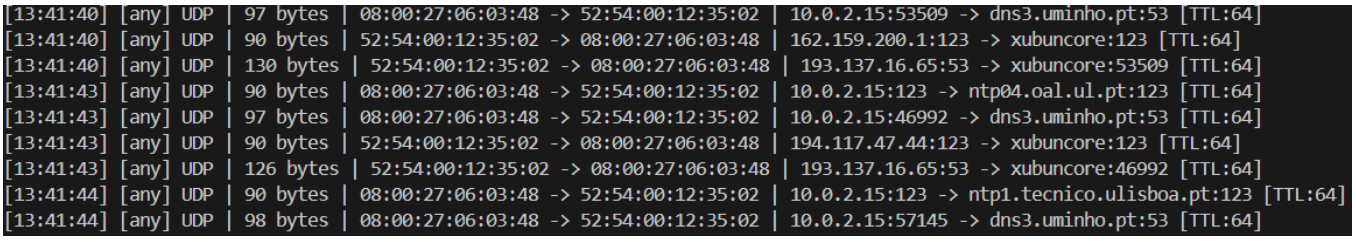


Figura 20 - *Exemplo de tráfego*

4.2 Modo Log (ficheiro)

O ficheiro Log é do tipo **txt**, que é constituído da seguinte maneira a hora que foi lançado o tipo de protocolo, o número de bytes enviados, em seguida temos o IP e a porta origem como também a do destino, o nome de domínio associado ao IP de destino e por último o TTL, como vemos na figura 20, temos o exemplo de tráfego UDP.

Tabela 2 *- Exemplo de Tráfego UDP*

Hora/Protocolo	Bytes Enviados	IP/Porta Origem	IP/Porta Destino	Time to Live
[20:48:33] UDP	90 bytes	10.0.2.15:123 -> 2.83.198.44:123	10.0.2.15:123 -> bl22-198-44.dsl.telepac.pt:123	[TTL:64]

4.3 Exportação Consolidada de Tráfego (`--save`)

Para além do modo log incremental descrito em 4.2, o sniffer disponibiliza um mecanismo alternativo de persistência através do argumento `--save <ficheiro>`, que produz um relatório consolidado da sessão de captura. Ao contrário do modo `-l`, em que cada pacote é escrito sequencialmente à medida que é capturado, o `--save` apenas gera o ficheiro no final da sessão, agregando metadados que não estão disponíveis durante a captura em curso (totais finais, débito médio, distribuição por protocolo).

O ficheiro gerado segue uma estrutura tripartida: um cabeçalho com data, hora, duração da sessão e totais agregados; uma lista completa dos pacotes capturados em formato textual idêntico ao do modo live, mas sem códigos de cor ANSI para garantir compatibilidade com qualquer editor de texto; e um bloco final com o resumo estatístico por protocolo. Esta organização torna o ficheiro adequado tanto para inspeção manual como

para processamento posterior por scripts de análise, sendo útil em cenários em que se pretende preservar a captura completa para integração em relatórios técnicos ou comparação entre sessões.

```
carolina-dias@carolinad:~/Secretária/RC/RC---TP2---PL4$ head -25 trafico_final.txt
================================================================
  PACKET SNIFFER — REGISTO DE TRÁFEGO
  Gerado em: 2026-05-05 09:37:50
  Total de pacotes: 314
================================================================

[09:37:16] TCP     |    66 bytes | 192.168.1.16:47706 -> 64.233.184.188:443 | 192.168.1.16:47706 -> wa-in-f188.1e100.net:443 (HTTPS) [Flags: A, TTL:64]
[09:37:16] TCP     |    66 bytes | 64.233.184.188:443 -> 192.168.1.16:47706 | 64.233.184.188:443 -> 192.168.1.16:47706 [Flags: A, TTL:116]
[09:37:16] UDP     |    85 bytes | 192.168.1.16:42773 -> 192.168.1.1:53 | 192.168.1.16:42773 -> 192.168.1.1:53 (DNS) [TTL:64]
[09:37:16] UDP     |   111 bytes | 192.168.1.1:53 -> 192.168.1.16:42773 | 192.168.1.1:53 -> 192.168.1.16:42773 [TTL:64]
[09:37:16] UDP     |    84 bytes | 192.168.1.16:45051 -> 192.168.1.1:53 | 192.168.1.16:45051 -> 192.168.1.1:53 (DNS) [TTL:64]
[09:37:16] UDP     |   109 bytes | 192.168.1.1:53 -> 192.168.1.16:45051 | 192.168.1.1:53 -> 192.168.1.16:45051 [TTL:64]
[09:37:18] ICMP    |    98 bytes | 192.168.1.16 -> 172.217.171.46 | Type 8 (TTL:64)
[09:37:18] ICMP    |    98 bytes | 172.217.171.46 -> 192.168.1.16 | Type 0 | RTT: 67.14 ms (TTL:115)
[09:37:19] ICMP    |    98 bytes | 192.168.1.16 -> 172.217.171.46 | Type 8 (TTL:64)
[09:37:19] ICMP    |    98 bytes | 172.217.171.46 -> 192.168.1.16 | Type 0 | RTT: 42.76 ms (TTL:115)
[09:37:20] ICMP    |    98 bytes | 192.168.1.16 -> 172.217.171.46 | Type 8 (TTL:64)
[09:37:20] ICMP    |    98 bytes | 172.217.171.46 -> 192.168.1.16 | Type 0 | RTT: 45.82 ms (TTL:115)
[09:37:21] ICMP    |    98 bytes | 192.168.1.16 -> 172.217.171.46 | Type 8 (TTL:64)
[09:37:21] ICMP    |    98 bytes | 172.217.171.46 -> 192.168.1.16 | Type 0 | RTT: 41.51 ms (TTL:115)
[09:37:21] UDP     |    73 bytes | 192.168.1.16:45149 -> 192.168.1.1:53 | 192.168.1.16:45149 -> 192.168.1.1:53 (DNS) [TTL:64]
[09:37:21] UDP     |    74 bytes | 192.168.1.16:34220 -> 192.168.1.1:53 | 192.168.1.16:34220 -> 192.168.1.1:53 (DNS) [TTL:64]
[09:37:21] UDP     |    77 bytes | 192.168.1.16:43328 -> 192.168.1.1:53 | 192.168.1.16:43328 -> 192.168.1.1:53 (DNS) [TTL:64]
[09:37:21] TCP     |    74 bytes | 192.168.1.16:40782 -> 104.16.124.96:443 | 192.168.1.16:40782 -> 104.16.124.96:443 (HTTPS) [Flags: S, TTL:64]
[09:37:22] UDP     |    70 bytes | 192.168.1.16:34671 -> 192.168.1.1:53 | 192.168.1.16:34671 -> 192.168.1.1:53 (DNS) [TTL:64]
```

Figura 21 *- Excerto do ficheiro consolidado gerado via `--save`, com cabeçalho, registo de pacotes e resumo final*

5. Funcionalidades Opcionais

Para além dos requisitos base definidos no enunciado, foram implementadas onze funcionalidades opcionais com o objetivo de enriquecer a experiência de utilização do sniffer e aprofundar a análise do tráfego capturado. Estas funcionalidades foram previamente discutidas e validadas com o docente das aulas práticas.

5.1 Estatísticas Finais de Captura

No final de cada sessão de captura, ao interromper o sniffer com `Ctrl+C`, é apresentado automaticamente no terminal um relatório de captura com recurso a cores ANSI, sem necessidade de instalar qualquer dependência adicional.

O relatório inclui as seguintes métricas globais da sessão: duração total, número total de pacotes capturados, volume total de dados em bytes e quilobytes, e débito médio em bytes por segundo. Adicionalmente, é apresentada uma tabela por protocolo identificado, com o número de pacotes, percentagem de pacotes relativamente ao total, volume de dados em bytes, percentagem de bytes e uma barra de distribuição visual proporcional.

A utilização de cores ANSI permite distinguir visualmente cada protocolo - `TCP` em azul, `UDP` em ciano, `ICMP` em verde e `ARP` em amarelo - tornando a leitura do relatório mais rápida e intuitiva, em especial em sessões com elevado volume de tráfego.

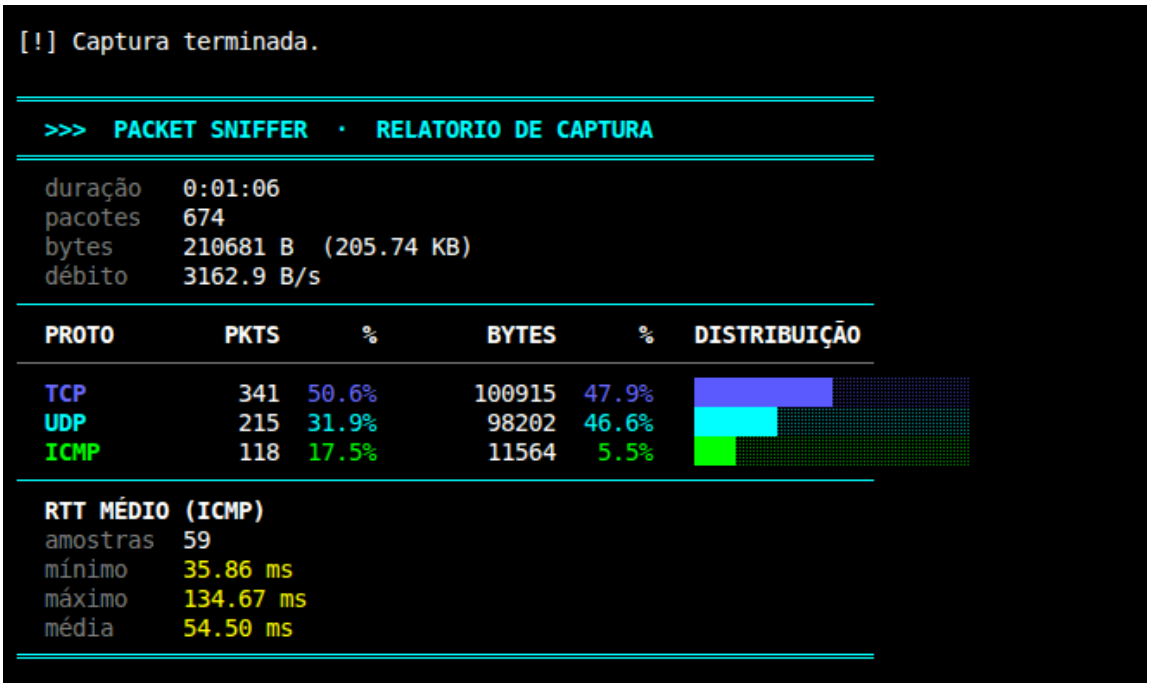


Figura 22 *- Relatório de estatísticas finais apresentado no terminal após Ctrl+C*

5.2 Cálculo de RTT Médio (ICMP)

O sniffer calcula automaticamente o RTT (Round-Trip Time) com base nos pares de pacotes ICMP echo request (Type 8) e echo reply (Type 0) capturados durante a sessão. Esta funcionalidade foi sugerida pelo docente das aulas práticas como métrica relevante para caracterizar o comportamento da rede observada.

O mecanismo de cálculo funciona da seguinte forma: quando um pacote ICMP echo request é capturado, o timestamp do pacote - fornecido pelo Scapy com precisão de microssegundos através do campo `pkt.time` - é guardado numa estrutura indexada pelo endereço IP de origem. Quando o echo reply correspondente é recebido, a diferença entre os dois timestamps é calculada e convertida para milissegundos, constituindo uma amostra de RTT.

No final da captura, o relatório apresenta o número de amostras recolhidas, o valor mínimo, máximo e a média aritmética do RTT em milissegundos. Esta informação permite caracterizar a latência da rede observada durante a sessão, complementando as métricas de volume já disponíveis.

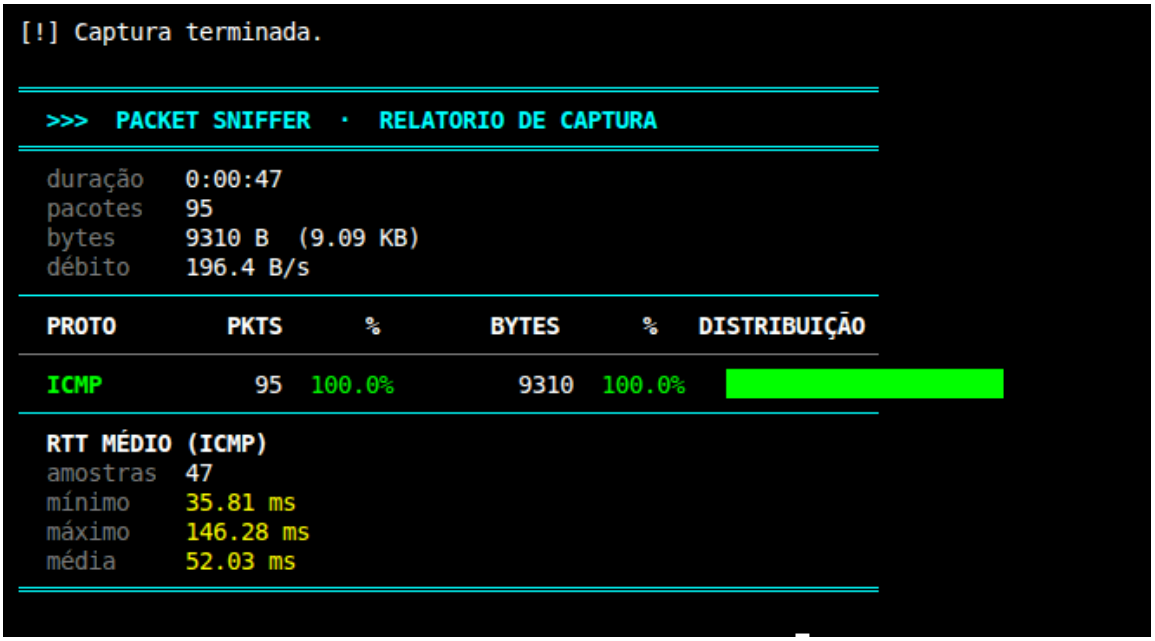


Figura 23 *- Secção de RTT Médio no relatório final, com amostras recolhidas via ping -4 google.com*

É importante notar que esta funcionalidade opera exclusivamente sobre tráfego ICMP em IPv4. Em redes com tráfego predominantemente IPv6, como redes domésticas modernas, os pacotes ICMPv6 são classificados como `UNKNOWN` pelo sniffer e não são considerados no cálculo do RTT. Esta constitui uma limitação conhecida, abordada na Secção 9.

5.3 Visualização Gráfica (--graph)

O sniffer suporta a geração automática de um gráfico com a distribuição de pacotes e bytes por protocolo, ativado através do argumento opcional `--graph` na linha de comandos. O gráfico é composto por dois painéis: um gráfico de barras verticais com o número de pacotes por protocolo e a respetiva percentagem, e um gráfico de barras horizontais com a distribuição percentual de bytes por protocolo.

O ficheiro é guardado automaticamente como `captura_stats.png` no diretório de execução do sniffer e, quando possível, aberto automaticamente no visualizador de imagens do sistema operativo.

Esta funcionalidade requer a instalação prévia da biblioteca `matplotlib`, podendo ser instalada com o comando `pip install matplotlib --break-system-packages`. A sua utilização é completamente opcional, uma vez que o sniffer funciona normalmente sem ela, apresentando uma mensagem informativa caso a biblioteca não esteja disponível. Esta decisão de design foi tomada deliberadamente para

garantir que o sniffer permanece funcional em qualquer ambiente sem dependências adicionais obrigatórias.

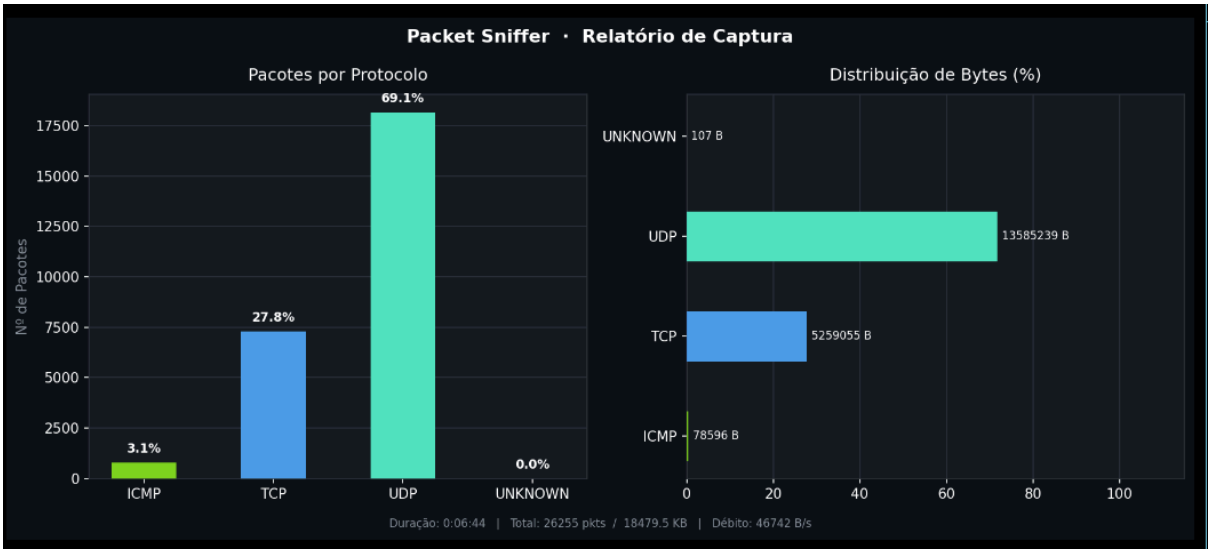


Figura 24 *- Gráfico gerado pelo sniffer com a distribuição de pacotes e bytes por protocolo*

5.4 Resolução Automática de *Hostnames* (DNS Resolver)

O sniffer resolve automaticamente os endereços IP de destino para os respetivos hostnames, utilizando a função `socket.gethostbyaddr()` da biblioteca padrão do Python, sem dependências externas. Para evitar latência excessiva em capturas com elevado volume de tráfego, os resultados são guardados numa cache interna: cada IP é resolvido apenas uma vez e reutilizado nas ocorrências seguintes.

No output do modo live, os pacotes TCP e UDP passam a mostrar o hostname do servidor de destino em vez do endereço IP numérico, tornando o tráfego muito mais legível e identificável. No final da captura, o relatório apresenta uma secção **`TOP HOSTNAMES RESOLVIDOS`** com os cinco hostnames mais frequentes da sessão, permitindo identificar rapidamente com que serviços o dispositivo comunicou.

Durante os testes na rede da Universidade do Minho foi possível observar hostnames como `dns3.uminho.pt`, servidores da Google (`tlv03s02-in-f14.1e100.net`), da Amazon AWS (`ec2-51-102-89-198.eu-central-1.c...`) e do GitHub (`lb-140-82-121-5-fra.github.com`), ilustrando bem a diversidade de serviços contactados durante uma sessão de navegação normal.

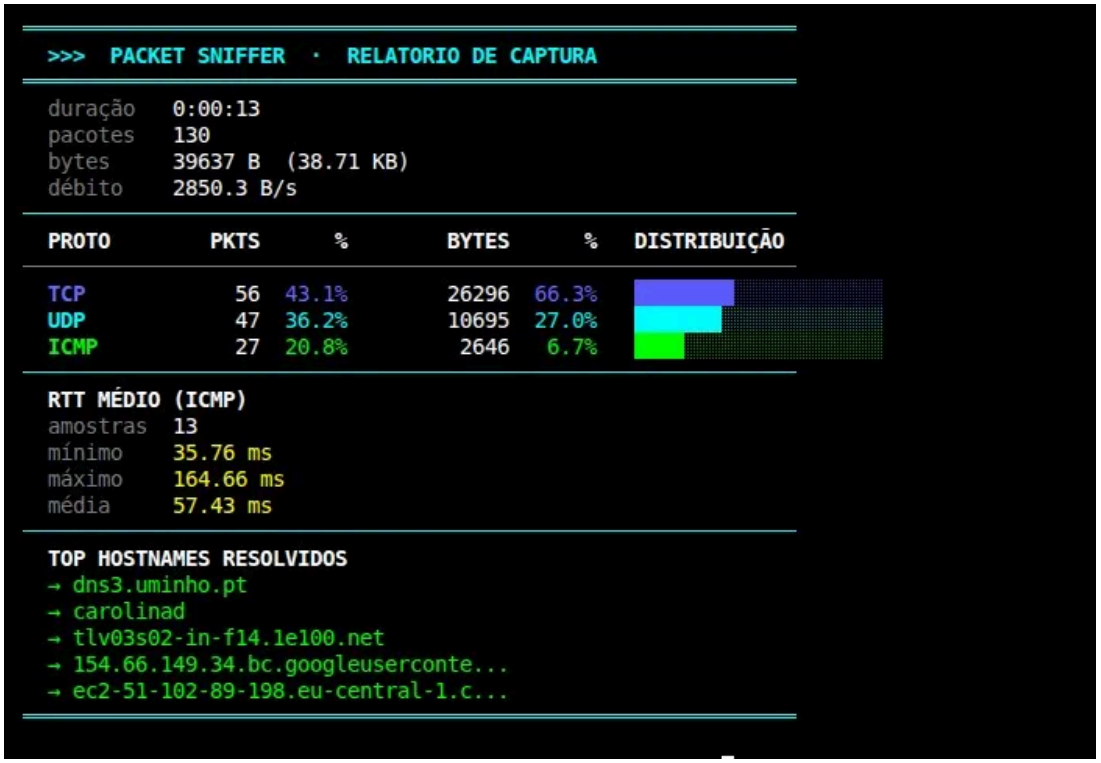


Figura 25 *- Secção TOP HOSTNAMES RESOLVIDOS no relatório final*

5.5 Deteção de Anomalias de Segurança

O sniffer monitoriza o tráfego em tempo real e emite alertas a vermelho no terminal quando detecta padrões suspeitos. Foram implementados dois mecanismos de deteção:

- **Port Scan**: O sniffer mantém um registo das portas TCP distintas que cada IP de origem tenta aceder. Se um único IP atingir 15 portas distintas ao longo da sessão, é gerado um alerta imediato no terminal e registado no relatório final. Este padrão é característico de ferramentas de reconhecimento de rede como o `nmap`.
- **ICMP Flood**: o sniffer contabiliza os pacotes ICMP echo request por IP de origem numa janela deslizante de 1 segundo. Se um IP enviar mais de 20 pings por segundo, é gerado um alerta de possível flood, indicador de um possível ataque de negação de serviço (DoS).

Para fins de demonstração e validação, o port scan foi simulado localmente com o comando `for port in $(seq 1 30); do nc -zv -w1 google.com $port 2>/dev/null; done`, que tenta estabelecer ligação a 30 portas distintas. Em ambiente real, o sniffer deteta este comportamento automaticamente de qualquer IP na rede, sem intervenção do utilizador.

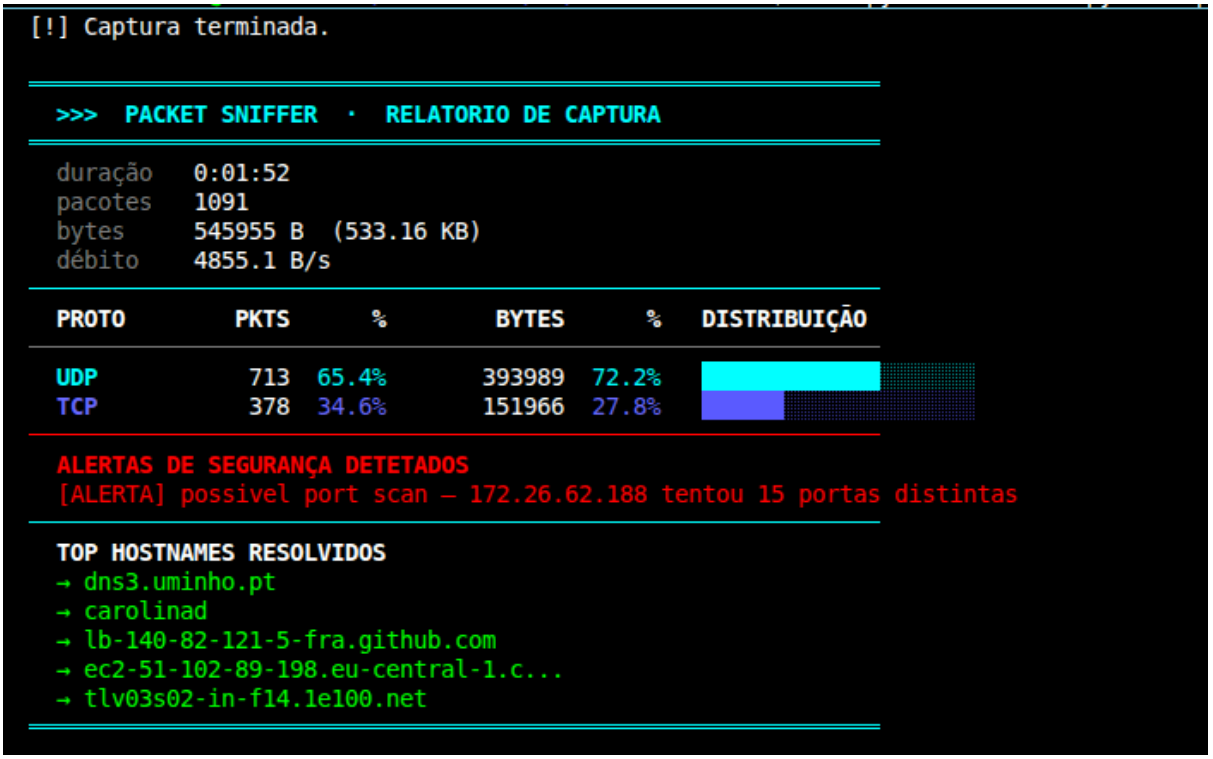


Figura 26 *- Alerta de port scan detetado em tempo real no terminal*

5.6 Captura com Duração Limitada (`--duration`)

O sniffer suporta a definição de uma duração máxima de captura através do argumento `--duration N`, em que N é o número de segundos. Ao atingir o tempo definido, a captura termina automaticamente e o relatório de estatísticas é apresentado, sem necessidade de intervenção manual com **`Ctrl+C`**.

Esta funcionalidade é especialmente útil em cenários de monitorização programada, como capturar exatamente 60 segundos de tráfego para análise comparativa, ou integrar o sniffer em scripts automatizados. Pode ser combinada com qualquer outro argumento, por exemplo:

```
sudo python3 sniffer.py -i wlp2s0 --live -l captura.txt --duration 30
```

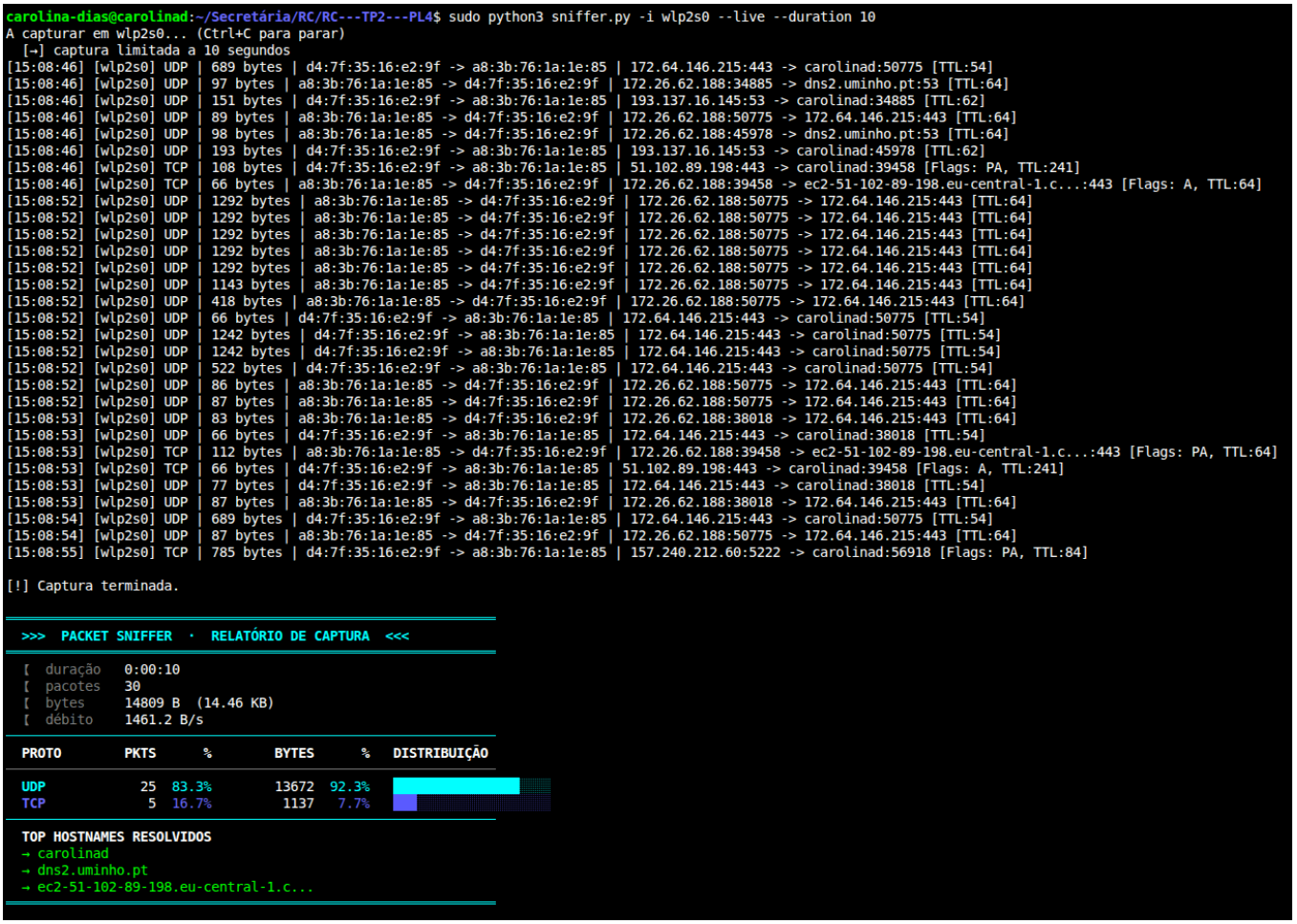


Figura 27 *- Captura com duração limitada a 10 segundos via `--duration`*

5.7 Identificação Automática de Serviços por Porta

Para enriquecer o output do sniffer e tornar a análise de tráfego mais imediata, foi implementado um mapeamento automático de portas TCP e UDP conhecidas para o nome do serviço aplicacional associado. Sempre que um pacote é processado, o sniffer consulta um dicionário interno (`PORT_SERVICES`) que regista 23 portas reservadas pela IANA, abrangendo serviços como HTTP (80), HTTPS (443), DNS (53), SSH (22), FTP (20/21), SMTP (25), POP3 (110), IMAP (143), DHCP (67/68), mDNS (5353), MySQL (3306) e PostgreSQL (5432), entre outros. Quando a porta de origem ou destino corresponde a uma entrada do dicionário, o nome do serviço é adicionado entre parêntesis junto ao número da porta no output da captura.

O efeito desta funcionalidade é particularmente visível em sessões com elevado volume de tráfego: em vez de o utilizador ter de associar mentalmente cada número de porta ao protocolo aplicacional correspondente, a informação é apresentada diretamente. Por exemplo, uma linha como `192.168.1.16:36062 -> 40.79.167.8:443 (HTTPS)`

permite identificar instantaneamente que se trata de tráfego web cifrado, sem necessidade de consulta externa.

O mapeamento é feito localmente sobre uma estrutura estática em memória, não introduzindo qualquer custo adicional de rede ou latência de processamento. Esta abordagem privilegia o desempenho em relação à exaustividade, o dicionário cobre apenas os serviços mais frequentemente observados em redes domésticas e académicas, sendo facilmente extensível caso se pretenda incluir protocolos adicionais.

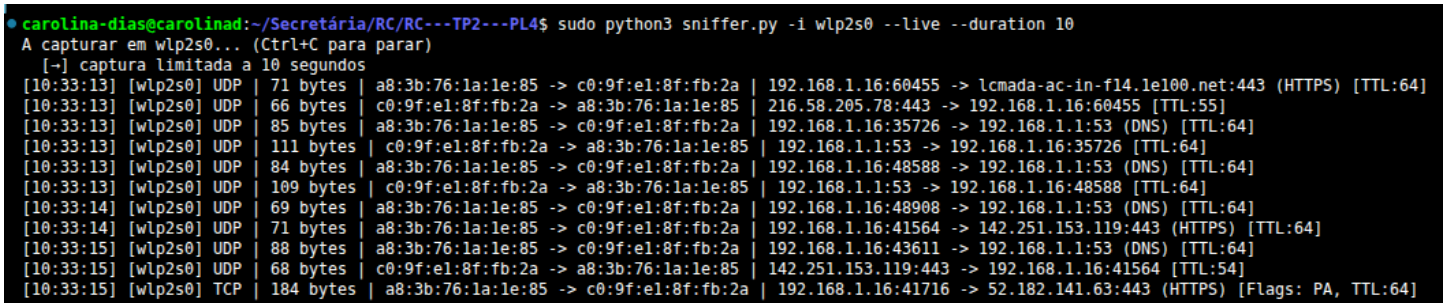


***Figura 28** - Identificação automática de serviços conhecidos junto às portas no output em modo live*

5.8 Suporte a Filtros BPF Nativos

O sniffer suporta filtros BPF (Berkeley Packet Filter) nativos através do argumento `--bpf`, com sintaxe idêntica à utilizada pelo `tcpdump` e pelo Wireshark. Ao contrário dos filtros pós-captura implementados em Python, os filtros BPF operam ao nível do kernel: a expressão é compilada e aplicada antes de o pacote atravessar o stack de rede, descartando à partida o tráfego que não interessa. Esta abordagem é significativamente mais eficiente em redes com elevado débito, sendo integrada com o Scapy através do parâmetro `filter` da função `sniff()`.

A linguagem BPF permite construir expressões compostas com elevada granularidade, por exemplo, `"tcp port 443"` para isolar tráfego HTTPS ou `"icmp or arp"` para observar protocolos de controlo. Esta flexibilidade aproxima o sniffer das ferramentas de referência da área, cumprindo um requisito explicitamente mencionado no enunciado.

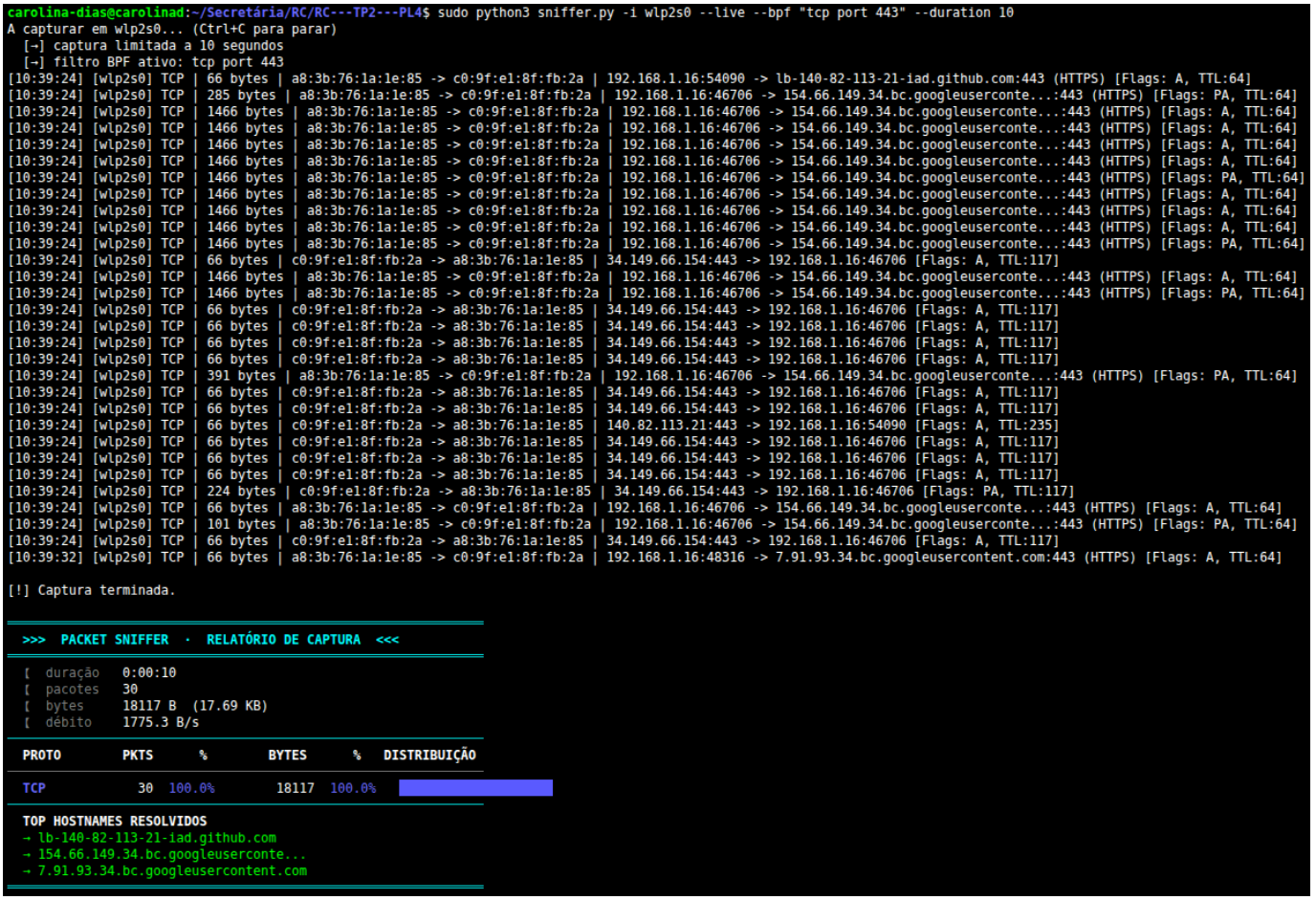


Figura 29 *- Filtro BPF nativo a restringir a captura*

5.9 Menu Interativo de Configuração e Análise

O sniffer disponibiliza, em alternativa à execução por argumentos de linha de comandos, um menu interativo acedido quando o programa é executado sem parâmetros. O menu apresenta opções numeradas que permitem configurar e iniciar a captura, consultar o tráfego com filtros aplicados, gerar gráficos, exportar o relatório final e terminar a sessão. A captura corre numa thread independente do menu, permitindo ao utilizador interagir com o sistema enquanto os pacotes continuam a ser processados em segundo plano.

A concorrência é gerida com o módulo `threading` da biblioteca padrão do Python: a thread principal mantém o ciclo do menu, enquanto uma thread secundária executa a função `sniff()` do Scapy. O estado da captura é exposto ao utilizador através de uma flag (EM CURSO / TERMINADA) visível no cabeçalho do menu, atualizada dinamicamente. Esta abordagem torna o sniffer mais acessível a utilizadores não familiarizados com argumentos de linha de comandos, sem comprometer a flexibilidade de execução por script.

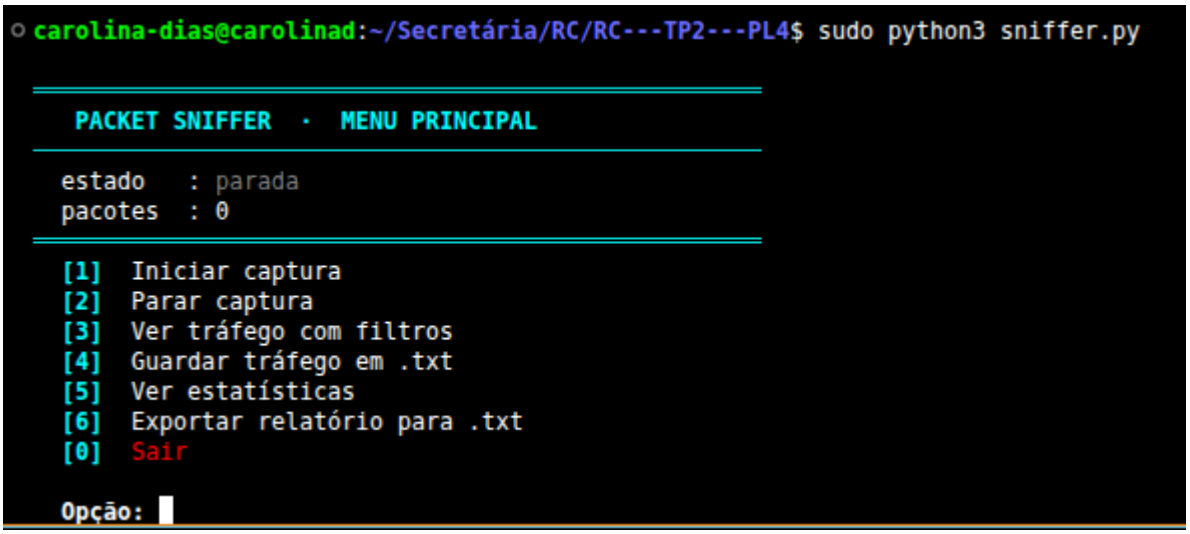


Figura 30 *- Menu interativo apresentado quando o sniffer é executado sem argumentos*

5.10 Estrutura Indexada para Filtragem Pós-Captura

Para suportar de forma eficiente as operações de filtragem aplicadas após a captura, o sniffer mantém uma hashtable em que cada pacote é indexado em múltiplos buckets: por protocolo, por endereço IP de origem e destino, e por porta de origem e destino. Esta indexação é construída incrementalmente à medida que os pacotes são processados, sem custo adicional significativo durante a captura.

Quando o utilizador aplica um filtro pós-captura, por exemplo, `--fproto TCP --fdport 443`, a pesquisa é feita por interseção de conjuntos sobre os buckets correspondentes, com complexidade próxima de O(1) por dimensão filtrada, em vez de uma travessia linear sobre toda a lista de pacotes. Em capturas com elevado volume, filtrar uma sessão com vários milhares de pacotes torna-se virtualmente instantâneo, comportamento que se aproxima do desempenho de ferramentas profissionais de análise de tráfego.

```
FILTRO DE TRÁFEGO

filtros   proto=TCP  dst_port=443
resultado 42 pacote(s) encontrado(s)

HORA       PROTO    BYTES  ORIGEM                 DESTINO                INFO
11:20:54   TCP         66  192.168.1.16:55692     18.97.36.68:443        192.168.1.16:55692 -> ec2-18-97-36-
11:20:54   TCP         94  192.168.1.16:55692     18.97.36.68:443        192.168.1.16:55692 -> ec2-18-97-36-
11:20:54   TCP        205  192.168.1.16:55692     18.97.36.68:443        192.168.1.16:55692 -> ec2-18-97-36-
11:20:54   TCP        179  192.168.1.16:58542     40.79.150.121:443      192.168.1.16:58542 -> 40.79.150.121
11:20:55   TCP       1494  192.168.1.16:58542     40.79.150.121:443      192.168.1.16:58542 -> 40.79.150.121
11:20:55   TCP       1080  192.168.1.16:58542     40.79.150.121:443      192.168.1.16:58542 -> 40.79.150.121
11:20:55   TCP         66  192.168.1.16:58542     40.79.150.121:443      192.168.1.16:58542 -> 40.79.150.121
11:20:56   TCP         94  192.168.1.16:37612     104.18.39.21:443       192.168.1.16:37612 -> 104.18.39.21:
11:20:57   TCP         66  192.168.1.16:40238     140.82.114.26:443      192.168.1.16:40238 -> lb-140-82-114
11:20:58   TCP         66  192.168.1.16:47706     64.233.184.188:443     192.168.1.16:47706 -> wa-in-f188.1e
11:21:00   TCP         66  192.168.1.16:39084     18.169.61.189:443      192.168.1.16:39084 -> ec2-18-169-61
11:21:01   TCP         66  192.168.1.16:48880     157.240.212.60:443     192.168.1.16:48880 -> whatsapp-cdn-
11:21:01   TCP        136  192.168.1.16:48880     157.240.212.60:443     192.168.1.16:48880 -> whatsapp-cdn-
11:21:01   TCP         66  192.168.1.16:48880     157.240.212.60:443     192.168.1.16:48880 -> whatsapp-cdn-
11:21:01   TCP        141  192.168.1.16:48880     157.240.212.60:443     192.168.1.16:48880 -> whatsapp-cdn-
11:21:01   TCP        163  192.168.1.16:48880     157.240.212.60:443     192.168.1.16:48880 -> whatsapp-cdn-
11:21:01   TCP         66  192.168.1.16:48880     157.240.212.60:443     192.168.1.16:48880 -> whatsapp-cdn-
11:21:02   TCP         66  192.168.1.16:52198     160.79.104.10:443      192.168.1.16:52198 -> 160.79.104.10
11:21:02   TCP         66  192.168.1.16:48202     34.149.66.154:443      192.168.1.16:48202 -> 154.66.149.34
11:21:04   TCP        890  192.168.1.16:44644     20.111.1.3:443         192.168.1.16:44644 -> 20.111.1.3:44
11:21:04   TCP         66  192.168.1.16:44644     20.111.1.3:443         192.168.1.16:44644 -> 20.111.1.3:44
11:21:05   TCP         66  192.168.1.16:39084     18.169.61.189:443      192.168.1.16:39084 -> ec2-18-169-61
11:21:05   TCP        957  192.168.1.16:38244     162.247.243.29:443     192.168.1.16:38244 -> 162.247.243.2
11:21:05   TCP         74  192.168.1.16:41458     162.247.243.29:443     192.168.1.16:41458 -> 162.247.243.2
11:21:05   TCP        827  192.168.1.16:38244     162.247.243.29:443     192.168.1.16:38244 -> 162.247.243.2
11:21:05   TCP         66  192.168.1.16:41458     162.247.243.29:443     192.168.1.16:41458 -> 162.247.243.2
11:21:05   TCP       1494  192.168.1.16:41458     162.247.243.29:443     192.168.1.16:41458 -> 162.247.243.2
11:21:05   TCP        650  192.168.1.16:41458     162.247.243.29:443     192.168.1.16:41458 -> 162.247.243.2
11:21:05   TCP         66  192.168.1.16:41458     162.247.243.29:443     192.168.1.16:41458 -> 162.247.243.2
11:21:05   TCP        130  192.168.1.16:41458     162.247.243.29:443     192.168.1.16:41458 -> 162.247.243.2
11:21:05   TCP       1118  192.168.1.16:41458     162.247.243.29:443     192.168.1.16:41458 -> 162.247.243.2
11:21:05   TCP         54  192.168.1.16:39084     18.169.61.189:443      192.168.1.16:39084 -> ec2-18-169-61
11:21:05   TCP         54  192.168.1.16:39084     18.169.61.189:443      192.168.1.16:39084 -> ec2-18-169-61
11:21:05   TCP         66  192.168.1.16:38244     162.247.243.29:443     192.168.1.16:38244 -> 162.247.243.2
11:21:05   TCP         78  192.168.1.16:41458     162.247.243.29:443     192.168.1.16:41458 -> 162.247.243.2
11:21:06   TCP        179  192.168.1.16:58542     40.79.150.121:443      192.168.1.16:58542 -> 40.79.150.121
11:21:06   TCP       1241  192.168.1.16:58542     40.79.150.121:443      192.168.1.16:58542 -> 40.79.150.121
11:21:06   TCP         66  192.168.1.16:58542     40.79.150.121:443      192.168.1.16:58542 -> 40.79.150.121
11:21:06   TCP         66  192.168.1.16:58542     40.79.150.121:443      192.168.1.16:58542 -> 40.79.150.121
11:21:06   TCP        101  192.168.1.16:58542     40.79.150.121:443      192.168.1.16:58542 -> 40.79.150.121
11:21:07   TCP        535  192.168.1.16:48880     157.240.212.60:443     192.168.1.16:48880 -> whatsapp-cdn-
11:21:07   TCP         66  192.168.1.16:48880     157.240.212.60:443     192.168.1.16:48880 -> whatsapp-cdn-

total  42 pacotes  /  13156 bytes  (12.85 KB)
```

Figura 31 *- Aplicação de filtros pós-captura através da hashtable indexada (resultado restrito a TCP/443)*

5.11 Dashboard Web para Visualização Interativa

Como complemento à interface de linha de comandos, foi desenvolvido um dashboard web em HTML, CSS e JavaScript que permite visualizar de forma interativa os dados típicos produzidos por uma sessão de captura. O topo da interface apresenta métricas globais em cards individuais: **total de pacotes capturados**, **volume de dados em kilobytes**, **número de alertas de segurança detetados** e um gauge animado para o **RTT médio ICMP** com indicação de mínimo, máximo e número de amostras. Abaixo das métricas são exibidas barras de distribuição por protocolo com codificação cromática consistente e a lista dos hostnames mais frequentemente contactados durante a sessão.

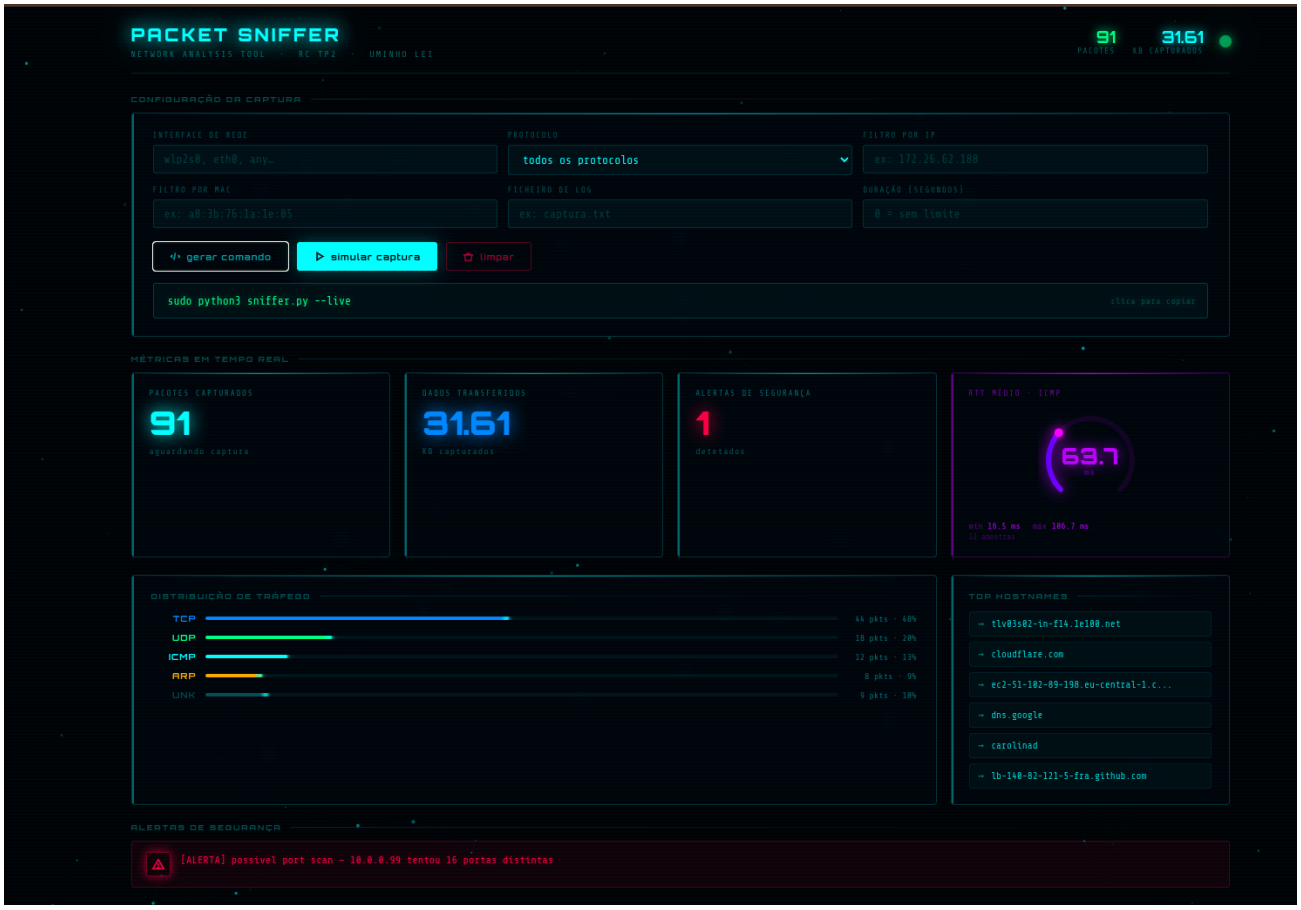


Figura 32 - Secção superior do dashboard

A metade inferior do dashboard inclui um painel de alertas de segurança com animação de flash aquando da deteção de anomalias, um registo de pacotes filtrável por protocolo e por palavra-chave, e uma secção de exportação que permite descarregar a captura em texto simples, em formato CSV ou como relatório consolidado. O painel de configuração no topo permite ainda gerar automaticamente o comando de execução do sniffer a partir dos parâmetros introduzidos.

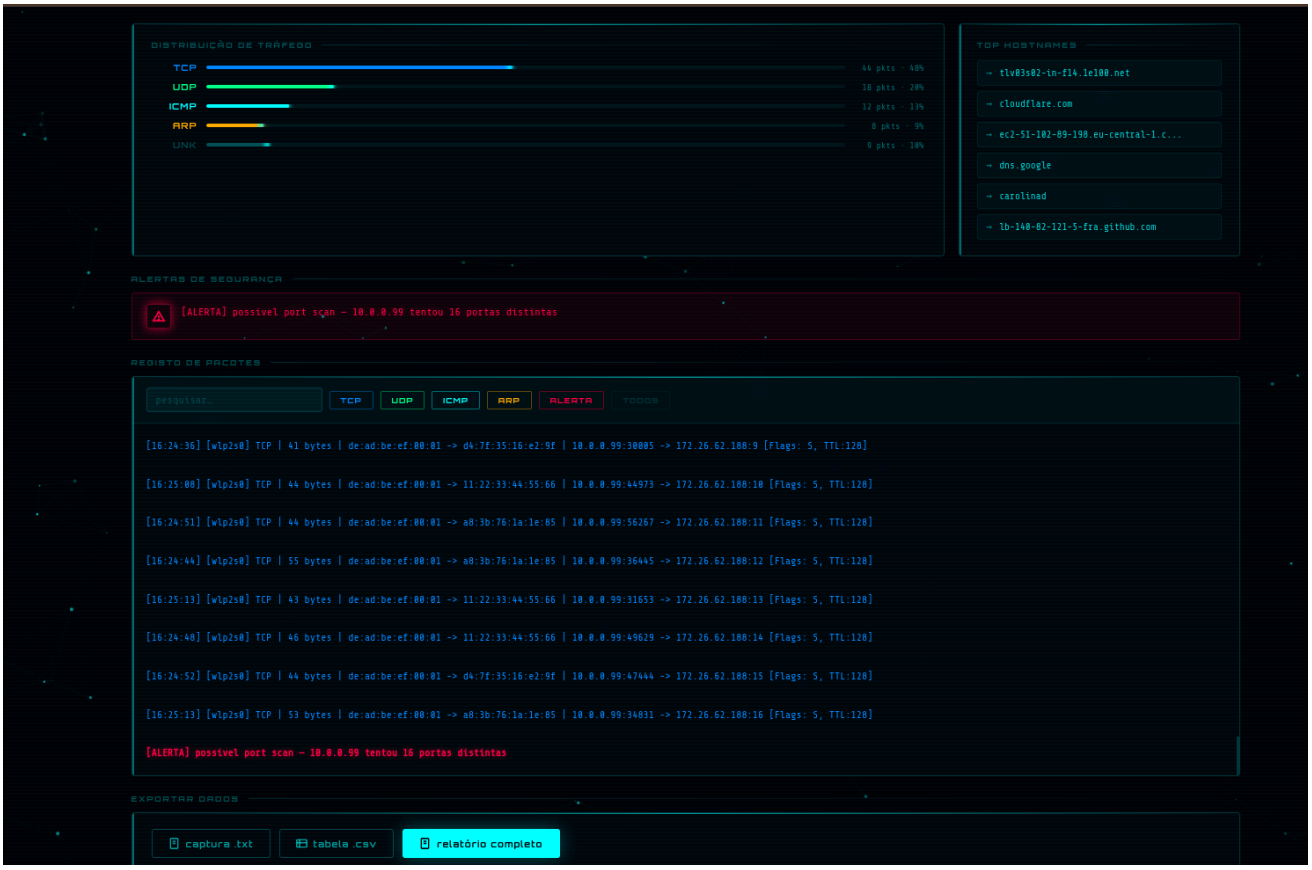


Figura 33 *- Secção inferior do dashboard*

Na versão atual, o dashbord opera de forma autónoma sobre dados de demonstração, não estando ligado em tempo real ao sniffer. A integração futura prevê o desenvolvimento de uma camada de servidor web que transmita os pacotes capturados via WebSocket, uma evolução que requer infraestrutura adicional fora do âmbito do presente trabalho.

6. Topologia CORE e Parte A

Para a validação em ambiente emulado, foi criada uma topologia no emulador CORE que permite desencadear e observar os protocolos identificados na Secção 2, esta tipologia é a mesma que foi a usada no primeiro trabalho.

6.1 Descrição da Topologia

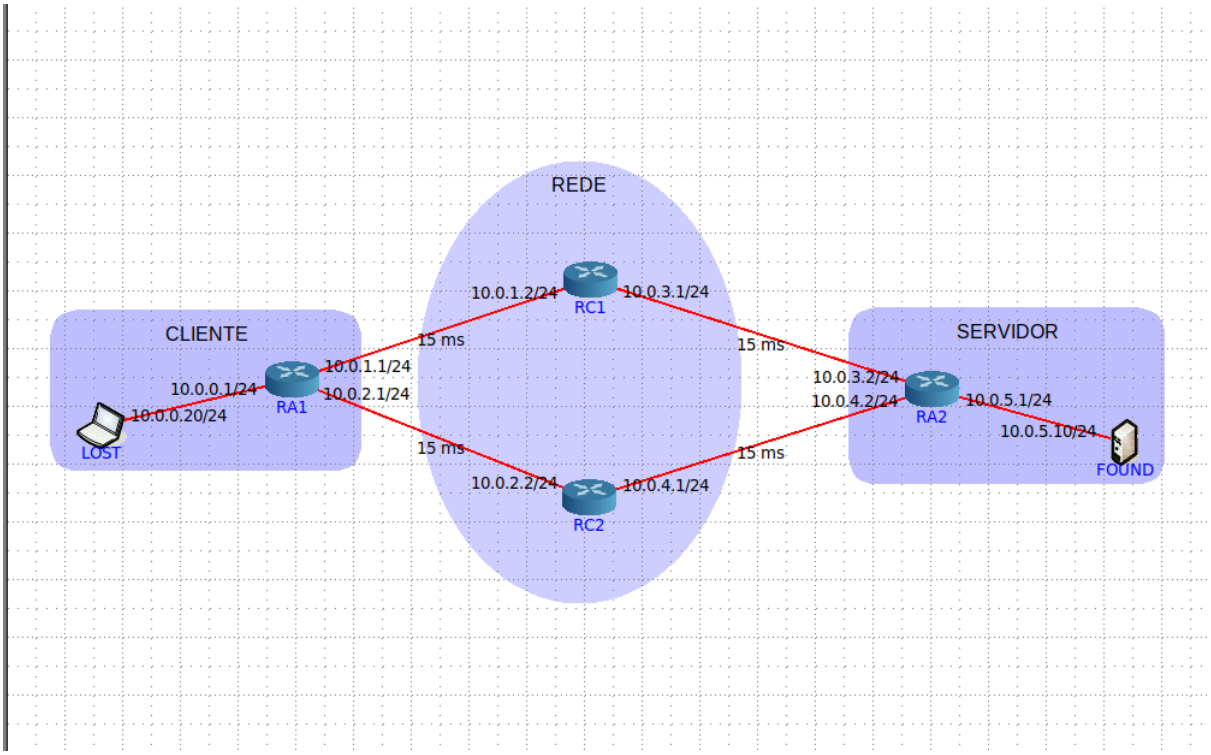


Figura 34 - *Topologia do programa core usada para testes*

A topologia é constituída da seguinte forma: um computador cliente chamado LOST está ligado a um router de acesso (RA1) com endereçamento 10.0.0.1/24 e 10.0.1.1/24, que por sua vez está ligado a dois routers de núcleo (RC1 e RC2). Estes estão interligados e ligados a um segundo router de acesso (RA2), que estabelece ligação ao servidor chamado FOUND com endereço 10.0.5.10/24. Todos os nós utilizam endereçamento IPv4 estático, com latências de 15 ms por trecho, o que corresponde a um RTT esperado de aproximadamente 60 ms entre LOST e FOUND.

6.2 Como a Topologia Permite Identificar os Protocolos

Ao testar o sniffer, foram executados no nó LOST os seguintes comandos para desencadear tráfego dirigido ao servidor FOUND, garantindo a observação dos protocolos pretendidos:

- `ping 10.0.5.10` - Gera tráfego ICMP echo request/reply, permitindo validar a deteção de mensagens de controlo e o cálculo de RTT;
- `nc -u 10.0.5.10 33434` - Utiliza o netcat em modo UDP para estabelecer uma sessão de transferência sobre UDP entre cliente e servidor;
- `nc 10.0.5.10 80` - Utiliza o netcat em modo TCP para estabelecer uma ligação TCP, permitindo observar o handshake de 3 vias (SYN, SYN-ACK, ACK), a transferência de dados (PSH+ACK) e a terminação ordenada (FIN, ACK);
- `traceroute 10.0.5.10` - Gera datagramas UDP com TTL incremental, expondo o caminho percorrido até ao destino e desencadeando respostas ICMP Time Exceeded em cada router intermédio;
- O tráfego ARP é gerado automaticamente pelo sistema operativo durante a primeira comunicação com cada nó, à medida que a tabela ARP é construída.

6.3 Capturas Obtidas e Testes

```
[20:49:29] UNKNOWN |    90 bytes | 00:00:00:aa:00:01 -> 33:33:00:00:00:05 | Ether / fe80::200:ff:feaa:1 > ff02::5
[20:49:29] UNKNOWN |    78 bytes | 10.0.0.1 -> 224.0.0.5 | Ether / 10.0.0.1 > 224.0.0.5 ospf / Raw
[20:49:29] ICMP    |    98 bytes | 10.0.0.20 -> 10.0.5.10 | Type 8 (TTL:64)
[20:49:29] ICMP    |    98 bytes | 10.0.5.10 -> 10.0.0.20 | Type 0 | RTT: 62.23 ms (TTL:61)
[20:49:29] ARP     |    42 bytes | 10.0.0.1 -> 10.0.0.20 | Request: 10.0.0.1 -> 10.0.0.20
[20:49:29] ARP     |    42 bytes | 10.0.0.20 -> 10.0.0.1 | Reply: 10.0.0.20 -> 10.0.0.1
[20:49:30] ICMP    |    98 bytes | 10.0.0.20 -> 10.0.5.10 | Type 8 (TTL:64)
[20:49:30] ICMP    |    98 bytes | 10.0.5.10 -> 10.0.0.20 | Type 0 | RTT: 62.00 ms (TTL:61)
[20:49:31] UNKNOWN |    78 bytes | 10.0.0.1 -> 224.0.0.5 | Ether / 10.0.0.1 > 224.0.0.5 ospf / Raw
[20:49:31] ICMP    |    98 bytes | 10.0.0.20 -> 10.0.5.10 | Type 8 (TTL:64)
[20:49:31] ICMP    |    98 bytes | 10.0.5.10 -> 10.0.0.20 | Type 0 | RTT: 62.01 ms (TTL:61)
[20:49:33] UNKNOWN |    70 bytes | 02:c2:01:47:65:fd -> 33:33:00:00:00:02 | Ether / IPv6 / ICMPv6ND_RS / ICMPv6 Ne
[20:49:33] UNKNOWN |    78 bytes | 10.0.0.1 -> 224.0.0.5 | Ether / 10.0.0.1 > 224.0.0.5 ospf / Raw
[20:49:35] UNKNOWN |    78 bytes | 10.0.0.1 -> 224.0.0.5 | Ether / 10.0.0.1 > 224.0.0.5 ospf / Raw
[20:49:36] UDP     |    74 bytes | 10.0.0.20:43208 -> 10.0.5.10:33434 | 10.0.0.20:43208 -> 10.0.5.10:33434 [TTL:1]
[20:49:36] ICMP    |   102 bytes | 10.0.0.1 -> 10.0.0.20 | Type 11 (TTL:64)
[20:49:36] UDP     |    74 bytes | 10.0.0.20:43276 -> 10.0.5.10:33435 | 10.0.0.20:43276 -> 10.0.5.10:33435 [TTL:1]
[20:49:36] ICMP    |   102 bytes | 10.0.0.1 -> 10.0.0.20 | Type 11 (TTL:64)
[20:49:36] UDP     |    74 bytes | 10.0.0.20:54164 -> 10.0.5.10:33436 | 10.0.0.20:54164 -> 10.0.5.10:33436 [TTL:1]
[20:49:36] ICMP    |   102 bytes | 10.0.0.1 -> 10.0.0.20 | Type 11 (TTL:64)
```

Figura 35 - *Resultado da captura de tráfego pelo CORE.*

Na figura acima, está o resultado de uma parte da captura feita pela rede criada pelo o core, ao usar o sniffer e os comandos ping, netstat -rn e traceroute, verificamos que o packet sniffer apanha vários protocolos como ICMP, UDP, ARP. Os pacotes UNKNOWN é uma limitação conhecida do ambiente CORE, onde os routers emulados executam OSPF para manutenção de tabelas de encaminhamento dinâmico, gerando tráfego que fica fora do âmbito de análise do sniffer e é corretamente sinalizado como UNKNOWN.

6.3.1 Testes

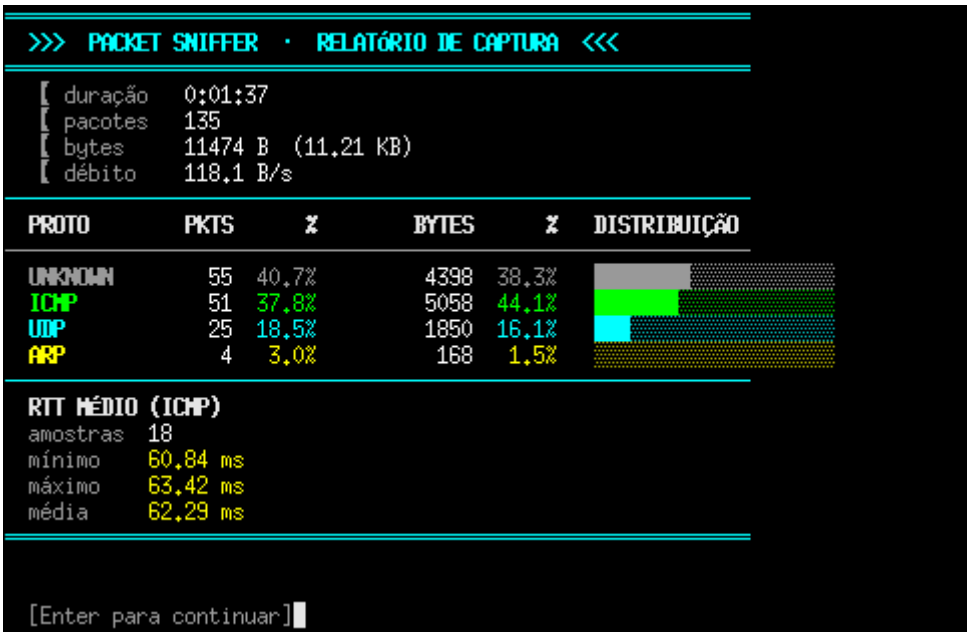


Figura 36 - Resultado dos teste na rede Core

O relatório de captura final da sessão em ambiente CORE registou 135 pacotes ao longo de 1 minuto e 37 segundos, totalizando 11 474 bytes (11.21 KB) com um débito médio de 118.1 B/s. Em termos de distribuição por protocolo, os pacotes UNKNOWN representaram a maioria com 40.7% (55 pacotes, 4 398 bytes), o que reflete o tráfego de controle OSPF dos routers emulados. O ICMP representou 37.8% (51 pacotes, 5 058 bytes), o UDP 18.5% (25 pacotes, 1 850 bytes) e o ARP 3.0% (4 pacotes, 168 bytes). O RTT médio calculado com base em 18 amostras ICMP foi de 62.29 ms, com mínimo de 60.84 ms e máximo de 63.42 ms, valores que confirmam a latência configurada na topologia. O elevado volume de pacotes UNKNOWN é uma limitação conhecida do ambiente CORE, onde os routers emulados executam OSPF para manutenção de tabelas de encaminhamento dinâmico, gerando tráfego que fica fora do âmbito de análise do sniffer e é corretamente sinalizado como UNKNOWN.

7. Testes e Validação

Esta secção descreve os testes efetuados para validar o correto funcionamento do sniffer, tanto no ambiente emulado como na interface real.

7.1 Testes em Contexto Real

Durante o teste foram usados, netcat, ping e netstat, para os servidores da google.com, numa rede que utiliza o serviço Vodafone.

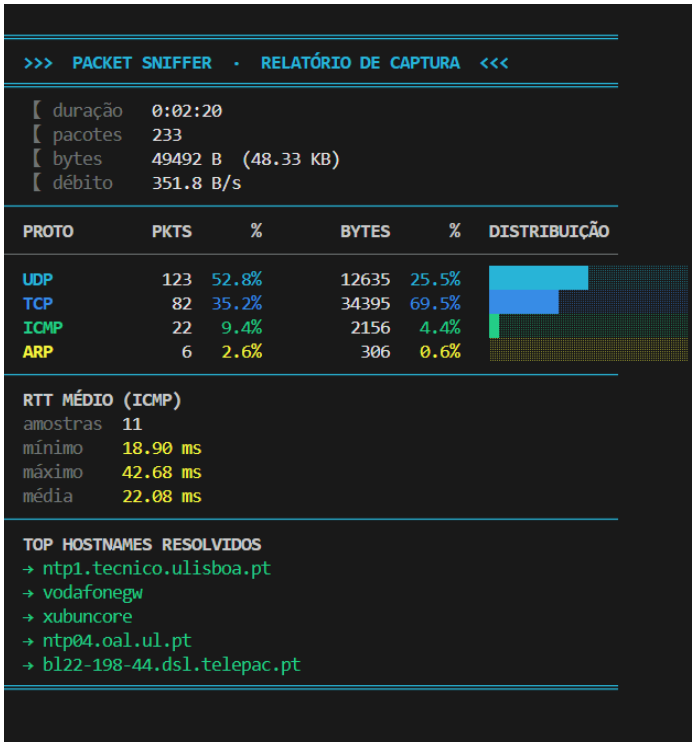


Figura 37 - *Relatório de captura*

Como se observa na captura, durante dois minutos foram obtidos 233 pacotes totalizando 49 492 bytes (48.33 KB), com um débito médio de 351.0 B/s. Em termos de distribuição por protocolo, o UDP foi dominante com 52.8% dos pacotes (123 pacotes, 12 635 bytes). O TCP representou 35.2% (82 pacotes, 34 395 bytes). O ICMP correspondeu a 9.4% (22 pacotes, 2 156 bytes), gerado pelos comandos ping google.com executados durante os testes. O ARP representou os restantes 2.6% (6 pacotes, 306 bytes), relativos à manutenção periódica da tabela ARP com o router local. O RTT médio calculado a partir de 11 amostras ICMP foi de 22.08 ms, com mínimo de 18.90 ms e máximo de 42.68 ms, valores consistentes com uma ligação de banda larga à infraestrutura Google. Os top hostnames resolvidos foram ntp1.tecnico.ulisboa.pt, vodafonegw, xubuncore, ntp04.oal.ul.pt e bl22-198-44.dsl.telepac.pt, revelando a diversidade de serviços contactados em segundo plano durante a sessão.

7.2 Testes de Filtros

```
FILTRO DE TRÁFEGO

filtros   proto=TCP
resultado 82 pacote(s) encontrado(s)

HORA      PROTO   BYTES  ORIGEM              DESTINO             INFO
21:06:43  TCP        74  10.0.2.15:41872     13.69.116.108:443   10.0.2.15:41872 -> 13.69.116.108:44
21:06:43  TCP        60  13.69.116.108:443   10.0.2.15:41872     13.69.116.108:443 -> xubuncore:4187
21:06:43  TCP        54  10.0.2.15:41872     13.69.116.108:443   10.0.2.15:41872 -> 13.69.116.108:44
21:06:43  TCP      1758  10.0.2.15:41872     13.69.116.108:443   10.0.2.15:41872 -> 13.69.116.108:44
21:06:43  TCP        60  13.69.116.108:443   10.0.2.15:41872     13.69.116.108:443 -> xubuncore:4187
21:06:43  TCP        60  13.69.116.108:443   10.0.2.15:41872     13.69.116.108:443 -> xubuncore:4187
21:06:43  TCP       153  13.69.116.108:443   10.0.2.15:41872     13.69.116.108:443 -> xubuncore:4187
21:06:43  TCP        54  10.0.2.15:41872     13.69.116.108:443   10.0.2.15:41872 -> 13.69.116.108:44
21:06:43  TCP       609  10.0.2.15:41872     13.69.116.108:443   10.0.2.15:41872 -> 13.69.116.108:44
21:06:43  TCP        60  13.69.116.108:443   10.0.2.15:41872     13.69.116.108:443 -> xubuncore:4187
21:06:43  TCP      2878  13.69.116.108:443   10.0.2.15:41872     13.69.116.108:443 -> xubuncore:4187
21:06:43  TCP      1514  13.69.116.108:443   10.0.2.15:41872     13.69.116.108:443 -> xubuncore:4187
21:06:43  TCP        54  10.0.2.15:41872     13.69.116.108:443   10.0.2.15:41872 -> 13.69.116.108:44
21:06:43  TCP      2406  13.69.116.108:443   10.0.2.15:41872     13.69.116.108:443 -> xubuncore:4187
```

Figura 38 *- O uso de filtro TCP após o fim da captura.*

Após a captura é usado um dos filtros possíveis, neste exemplo foi usado o filtro TCP. O filtro devolveu 82 pacotes correspondentes exclusivamente a tráfego TCP, confirmando a correta indexação e filtragem na hashtable interna. O resultado confirma que o mecanismo de filtragem pós-captura devolve corretamente apenas os pacotes correspondentes ao critério aplicado, preservando a ordem temporal original.

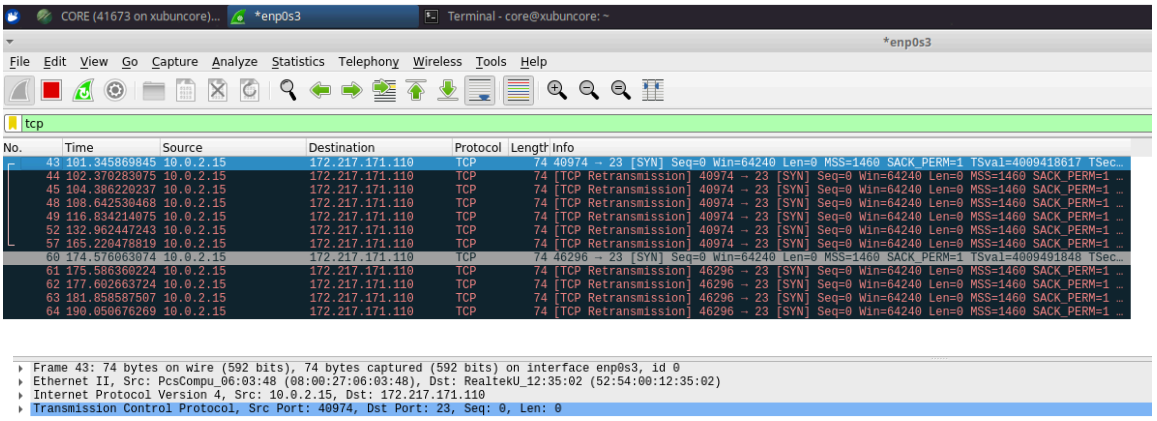


Figura 39 - *Captura Wireshark do protocolo TCP*

Para validar a correção dos filtros aplicados, a mesma sessão foi também analisada com o Wireshark, ferramenta de referência da área. Aplicando o filtro `tcp` no Wireshark e comparando com o resultado obtido pelo sniffer através de `--fproto TCP`, verificou-se que ambos identificam o mesmo conjunto de pacotes, com timestamps, endereços e portas coincidentes. Esta correspondência confirma que a classificação de protocolos e a filtragem pós-captura na hashtable interna estão a funcionar corretamente, alinhando o comportamento do sniffer com o das ferramentas profissionais já estabelecidas.

7.3 Testes de Logging

```
59   [21:07:05] UDP     |    81 bytes | 10.0.2.15:53181 -> 192.168.1.1:53 | 10.0.2.15:53181 -> vodafonegw:53 [TTL:64]
60   [21:07:05] UDP     |    81 bytes | 10.0.2.15:37915 -> 192.168.1.1:53 | 10.0.2.15:37915 -> vodafonegw:53 [TTL:64]
61   [21:07:05] UDP     |    97 bytes | 192.168.1.1:53 -> 10.0.2.15:53181 | 192.168.1.1:53 -> xubuncore:53181 [TTL:64]
62   [21:07:05] UDP     |   109 bytes | 192.168.1.1:53 -> 10.0.2.15:37915 | 192.168.1.1:53 -> xubuncore:37915 [TTL:64]
63   [21:07:05] ICMP    |    98 bytes | 10.0.2.15 -> 216.58.205.174 | Type 8 (TTL:64)
64   [21:07:05] ICMP    |    98 bytes | 216.58.205.174 -> 10.0.2.15 | Type 0 | RTT: 20.18 ms (TTL:116)
65   [21:07:05] UDP     |    98 bytes | 10.0.2.15:49182 -> 192.168.1.1:53 | 10.0.2.15:49182 -> vodafonegw:53 [TTL:64]
66   [21:07:05] UDP     |   168 bytes | 192.168.1.1:53 -> 10.0.2.15:49182 | 192.168.1.1:53 -> xubuncore:49182 [TTL:64]
67   [21:07:06] ARP     |    42 bytes | 10.0.2.15 -> 10.0.2.2 | Request: 10.0.2.15 -> 10.0.2.2
68   [21:07:06] ARP     |    60 bytes | 10.0.2.2 -> 10.0.2.15 | Reply: 10.0.2.2 -> 10.0.2.15
69   [21:07:06] ICMP    |    98 bytes | 10.0.2.15 -> 216.58.205.174 | Type 8 (TTL:64)
70   [21:07:06] ICMP    |    98 bytes | 216.58.205.174 -> 10.0.2.15 | Type 0 | RTT: 19.22 ms (TTL:116)
71   [21:07:07] ICMP    |    98 bytes | 10.0.2.15 -> 216.58.205.174 | Type 8 (TTL:64)
72   [21:07:07] ICMP    |    98 bytes | 216.58.205.174 -> 10.0.2.15 | Type 0 | RTT: 18.90 ms (TTL:116)
73   [21:07:08] ICMP    |    98 bytes | 10.0.2.15 -> 216.58.205.174 | Type 8 (TTL:64)
74   [21:07:08] ICMP    |    98 bytes | 216.58.205.174 -> 10.0.2.15 | Type 0 | RTT: 21.45 ms (TTL:116)
```

Figura 40 - *O ficheiro log esperado*

O ficheiro de log gerado pelo modo -l foi inspecionado para validar a correta persistência dos dados. Como se observa na figura 41, cada linha contém todos os campos esperados: timestamp no formato [HH:MM:SS], protocolo, tamanho em bytes, IP e porta de origem, IP e porta de destino com hostname resolvido, e valor de TTL. A figura mostra exemplos de várias linhas UDP com resolução de hostname para vodafonegw:53 e xubuncore:37915, linhas ICMP com cálculo de RTT (ex: Type 0 | RTT: 20.18 ms (TTL:116)) e linhas ARP com Request e Reply, confirmando que os três tipos de tráfego são corretamente serializados para ficheiro.

8. Análise na Interface Real (*Parte B*)

Nesta fase, o sniffer foi executado sobre a interface de rede real do computador, permitindo observar tráfego real e protocolos não presentes no ambiente emulado.

8.1 Configuração e Setup

O sniffer foi executado num computador com sistema operativo Linux (Ubuntu), sobre a interface de rede sem fios `wlp2s0`, com endereço `IP 172.20.10.3/28`. A captura de pacotes em interface real requer permissão do administrador, pelo que o sniffer foi iniciado com `sudo`. O comando utilizado foi:

```
sudo python3 sniffer.py -i wlp2s0 --live -l captura.txt
```

Os modos `--live` e `-l` foram ativados em simultâneo, permitindo visualizar os pacotes em tempo real na consola e guardar a captura no ficheiro **captura.txt** para análise posterior.

```
carolina-dias@carolinad:~/Secretária/RC/RC---TP2---PL4$ sudo python3 sniffer.py -i wlp2s0 --live -l captura.txt
A iniciar captura...
[11:33:44] UDP | 276 bytes | 142.250.74.174:443 -> 192.168.1.20:36509 [TTL:59]
[11:33:44] UDP | 78 bytes | 142.250.74.174:443 -> 192.168.1.20:36509 [TTL:59]
[11:33:44] UDP | 76 bytes | 192.168.1.20:36509 -> 142.250.74.174:443 [TTL:64]
[11:33:44] UDP | 840 bytes | 192.168.1.20:36509 -> 142.250.74.174:443 [TTL:64]
[11:33:44] UDP | 75 bytes | 142.250.74.174:443 -> 192.168.1.20:36509 [TTL:59]
[11:33:44] UDP | 75 bytes | 192.168.1.20:36509 -> 142.250.74.174:443 [TTL:64]
[11:33:44] UDP | 212 bytes | 192.168.1.5:57576 -> 239.255.255.250:1900 [TTL:1]
[11:33:44] ICMP | 98 bytes | Type 8 (TTL:64)
[11:33:44] ICMP | 98 bytes | Type 0 (TTL:117)
[11:33:44] UDP | 1057 bytes | 142.250.74.174:443 -> 192.168.1.20:36509 [TTL:59]
[11:33:44] UDP | 82 bytes | 192.168.1.20:36509 -> 142.250.74.174:443 [TTL:64]
[11:33:44] UDP | 68 bytes | 142.250.74.174:443 -> 192.168.1.20:36509 [TTL:59]
[11:33:45] UDP | 212 bytes | 192.168.1.5:57576 -> 239.255.255.250:1900 [TTL:1]
[11:33:45] ICMP | 98 bytes | Type 8 (TTL:64)
[11:33:45] ICMP | 98 bytes | Type 0 (TTL:117)
[11:33:45] TCP | 66 bytes | 192.168.1.20:48748 -> 35.190.87.184:443 [Flags: A, TTL:64]
[11:33:45] TCP | 66 bytes | 35.190.87.184:443 -> 192.168.1.20:48748 [Flags: A, TTL:122]
[11:33:45] TCP | 139 bytes | 142.251.25.94:443 -> 192.168.1.20:52528 [Flags: PA, TTL:122]
[11:33:45] TCP | 66 bytes | 192.168.1.20:52528 -> 142.251.25.94:443 [Flags: FA, TTL:64]
[11:33:45] TCP | 66 bytes | 142.251.25.94:443 -> 192.168.1.20:52528 [Flags: FA, TTL:122]
[11:33:45] TCP | 66 bytes | 192.168.1.20:52528 -> 142.251.25.94:443 [Flags: A, TTL:64]
[11:33:46] ICMP | 98 bytes | Type 8 (TTL:64)
[11:33:46] UDP | 212 bytes | 192.168.1.5:57576 -> 239.255.255.250:1900 [TTL:1]
[11:33:46] ICMP | 98 bytes | Type 0 (TTL:117)
[11:33:47] ICMP | 98 bytes | Type 8 (TTL:64)
[11:33:47] UDP | 212 bytes | 192.168.1.5:57576 -> 239.255.255.250:1900 [TTL:1]
[11:33:47] ICMP | 98 bytes | Type 0 (TTL:117)
[11:33:48] ICMP | 98 bytes | Type 8 (TTL:64)
[11:33:48] ICMP | 98 bytes | Type 0 (TTL:117)
[11:33:48] TCP | 179 bytes | 192.168.1.20:55108 -> 52.178.17.2:443 [Flags: PA, TTL:64]
[11:33:48] TCP | 1494 bytes | 192.168.1.20:55108 -> 52.178.17.2:443 [Flags: A, TTL:64]
[11:33:48] TCP | 1305 bytes | 192.168.1.20:55108 -> 52.178.17.2:443 [Flags: PA, TTL:64]
[11:33:48] TCP | 66 bytes | 52.178.17.2:443 -> 192.168.1.20:55108 [Flags: A, TTL:109]
[11:33:48] TCP | 105 bytes | 52.178.17.2:443 -> 192.168.1.20:55108 [Flags: PA, TTL:109]
```

Figura 41 *- Comando de execução do sniffer na interface real `wlp2s0`*

8.2 Protocolos Observados

Durante a captura foram identificados os seguintes protocolos, previamente validados no ambiente CORE:

```
carolina-dias@carolinad:~/Secretária/RC/RC---TP2---PL4$ grep "ICMP" captura.txt | head -5
[00:37:35] ICMP | 98 bytes | Type 8 (TTL:64)
[00:37:35] ICMP | 98 bytes | Type 0 (TTL:111)
[00:37:36] ICMP | 98 bytes | Type 8 (TTL:64)
[00:37:36] ICMP | 98 bytes | Type 0 (TTL:111)
[00:37:37] ICMP | 98 bytes | Type 8 (TTL:64)
carolina-dias@carolinad:~/Secretária/RC/RC---TP2---PL4$
```

Figura 42 *- Captura de pacotes ICMP*

Foram capturados pares de pacotes ICMP Type 8 (echo request) e Type 0 (echo reply), gerados pelo comando `ping google.com` executado em paralelo. O TTL de 64 indica origem local e o TTL de 111 corresponde à resposta do servidor remoto, refletindo o número de saltos efectuados.

```
carolina-dias@carolinad:~/Secretária/RC/RC---TP2---PL4$ grep "ARP" captura.txt
[00:37:26] ARP | 42 bytes | Request: 172.20.10.1 -> 172.20.10.3
[00:37:26] ARP | 42 bytes | Reply: 172.20.10.3 -> 172.20.10.1
[00:39:03] ARP | 42 bytes | Request: 172.20.10.1 -> 172.20.10.3
[00:39:03] ARP | 42 bytes | Reply: 172.20.10.3 -> 172.20.10.1
carolina-dias@carolinad:~/Secretária/RC/RC---TP2---PL4$
```

Figura 43 - *Captura de pacotes ARP*

O router (`172.20.10.1`) enviou ARP Requests à máquina local (`172.20.10.3`), que respondeu com ARP Replies. Este comportamento é esperado e corresponde ao mecanismo de resolução de endereços MAC na rede local, ocorrendo periodicamente para manter a tabela ARP atualizada.

```
carolina-dias@carolinad:~/Secretária/RC/RC---TP2---PL4$ grep "TCP" captura.txt | head -5
[00:37:11] TCP | 494 bytes | 172.20.10.3:45044 -> 40.79.167.8:443 [Flags: PA, TTL:64]
[00:37:11] TCP | 1464 bytes | 172.20.10.3:45044 -> 40.79.167.8:443 [Flags: A, TTL:64]
[00:37:11] TCP | 1158 bytes | 172.20.10.3:45044 -> 40.79.167.8:443 [Flags: PA, TTL:64]
[00:37:11] TCP | 66 bytes | 40.79.167.8:443 -> 172.20.10.3:45044 [Flags: A, TTL:103]
[00:37:11] TCP | 66 bytes | 162.247.241.14:443 -> 172.20.10.3:43200 [Flags: A, TTL:54]
carolina-dias@carolinad:~/Secretária/RC/RC---TP2---PL4$
```

Figura 44 - *Captura de pacotes TCP com identificação de flags em ligações HTTPS*

Observa-se tráfego TCP com múltiplas flags: `PA` (Push+ACK, envio de dados), `A` (ACK, confirmação de receção), `SA` (SYN-ACK, parte do handshake de 3 vias) e `FA` (FIN+ACK, terminação de ligação). A maior parte do tráfego usa a porta 443, correspondente a ligações HTTPS estabelecidas pelo sistema operativo em segundo plano.

```
carolina-dias@carolinad:~/Secretária/RC/RC---TP2---PL4$ grep "UDP" captura.txt | head -5

[00:37:11] UDP | 103 bytes | 172.20.10.3:33779 -> 172.20.10.1:53 [TTL:64]
[00:37:11] UDP | 103 bytes | 172.20.10.3:42044 -> 172.20.10.1:53 [TTL:64]
[00:37:11] UDP | 223 bytes | 172.20.10.1:53 -> 172.20.10.3:42044 [TTL:64]
[00:37:11] UDP | 268 bytes | 172.20.10.1:53 -> 172.20.10.3:33779 [TTL:64]
[00:37:11] UDP | 113 bytes | 172.20.10.3:60481 -> 172.20.10.1:53 [TTL:64]
carolina-dias@carolinad:~/Secretária/RC/RC---TP2---PL4$
```

Figura 45 - *Captura de tráfego UDP com queries DNS para o servidor local*

O tráfego UDP identificado inclui maioritariamente queries DNS, reconhecíveis pelo porto de destino 53, sendo `172.20.10.1` o servidor DNS local (router). Estas queries são geradas automaticamente pelo sistema durante a resolução de nomes de domínio.

8.3 Protocolos Extra (não presentes no CORE)

Para além dos protocolos já observados no ambiente emulado, foi identificado na interface real tráfego UDP com destino à porta 443, nomeadamente para os endereços `160.79.104.10` e `52.97.117.2`. Este padrão corresponde ao protocolo **QUIC** (Quick UDP Internet Connections), utilizado por serviços modernos como alternativa mais eficiente ao TCP/TLS. O endereço `160.79.104.10` pertence à infraestrutura da Google, sendo associado a serviços como o YouTube ou Google Meet, enquanto o endereço `52.97.117.2` pertence à Microsoft, provavelmente associado a serviços como o OneDrive ou Microsoft Teams. O QUIC opera sobre UDP e oferece fiabilidade e segurança de forma mais eficiente que o TCP clássico, sendo gerado automaticamente por aplicações em segundo plano. Este protocolo não é facilmente reproduzível no ambiente CORE, sendo apenas observável em tráfego real.

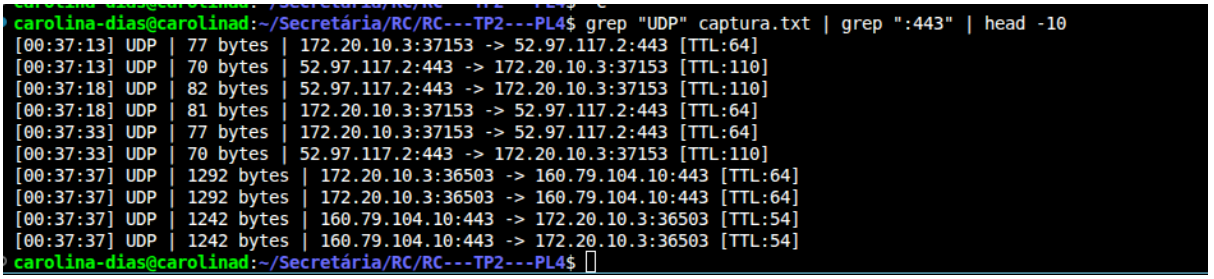


Figura 46 *- Tráfego UDP/443 identificado como protocolo QUIC, gerado por serviços Google*

8.4 Obstáculos Encontrados

Durante a execução do sniffer na interface real foram encontrados os seguintes obstáculos:

- O primeiro obstáculo foi a instalação do Scapy com permissões de root. Ao correr o sniffer com `sudo`, o Python utilizado era diferente do ambiente do utilizador, pelo que o módulo Scapy não era reconhecido. A solução foi instalar a biblioteca explicitamente para o Python do sistema com o comando `sudo pip3 install scapy --break-system-packages`;

- O segundo obstáculo foi o volume elevado de pacotes capturados. Ao contrário do ambiente CORE, onde o tráfego é controlado e previsível, a interface real captura todo o tráfego de fundo gerado pelo sistema operativo (atualizações, sincronizações, notificações), tornando difícil isolar os pacotes relevantes. A utilização dos filtros por protocolo (`-p`) e do ficheiro de log com `grep` permitiu contornar este problema.

9. Limitações da Aplicação

Apesar das funcionalidades implementadas, o sniffer apresenta as seguintes limitações conhecidas:

- **Permissões de sistema:** A captura em interface real requer permissões de administrador (`sudo`), uma vez que o acesso direto às interfaces de rede é restrito pelo sistema operativo. Em sistemas sem permissões de root, a execução falha silenciosamente ou com erro de socket.

- **Tráfego cifrado:** O sniffer não suporta a desencriptação de tráfego cifrado (e.g., TLS/HTTPS, QUIC com payload cifrado). Os pacotes são capturados e classificados ao nível do transporte, mas o conteúdo das camadas superiores não é interpretado.

- **ICMPv6 parcialmente suportado:** Alguns subtipos de mensagens ICMPv6 (em particular os tipos menos comuns, como Multicast Listener Discovery e Router Advertisement) são classificados como `UNKNOWN` pelo parser atual, não sendo descodificados nem apresentados com os campos específicos do protocolo.

- **Latência introduzida pelo DNS resolver:** A funcionalidade de resolução inversa de endereços IP para nomes de domínio pode introduzir latência visível em capturas de alto volume, especialmente quando o servidor DNS não responde dentro do tempo esperado ou quando a rede não tem conectividade DNS.

- **Thresholds de deteção de anomalias fixos:** A deteção de comportamentos suspeitos descrita na **Secção 5.5** (port scanning com mais de 15 portas distintas, ICMP flooding com mais de 20 pacotes/s) utiliza valores fixos definidos no código.

- **Dashboard com dados simulados:** O dashboard HTML descrito na **Secção 5.11** apresenta dados de exemplo pré-definidos, não refletindo em tempo real os dados da sessão de captura ativa.

- **Modo promíscuo condicionado:** A captura em modo promíscuo (que permitiria ver tráfego não endereçado à máquina) depende do suporte da interface de rede e das permissões do sistema. Em interfaces sem fios com drivers que não suportam este modo, a captura fica limitada ao tráfego dirigido ao host.

- **Sem funcionalidades ativas:** Por restrição ética e do enunciado, o sniffer opera exclusivamente em modo passivo. Funcionalidades como injeção de pacotes, ataques man-in-the-middle ou de autenticação não são suportadas nem previstas.

10. Conclusão

Este trabalho consistiu no desenvolvimento de um packet sniffer em Python com a biblioteca Scapy, capaz de capturar e classificar tráfego em interfaces emuladas e reais. Todos os requisitos do enunciado foram cumpridos, tanto na identificação dos protocolos ARP, ICMP, TCP e UDP com análise dos respetivos cabeçalhos, filtros por protocolo, IP, MAC e expressão BPF, e modos live e log com possibilidade de utilização simultânea. Foram ainda desenvolvidas onze funcionalidades opcionais validadas com o docente, descritas em detalhe na Secção 5.

A parte mais formativa do trabalho foi a comparação entre os dois ambientes de captura. No CORE tudo é previsível e controlável, o que ajudou a validar o sniffer com confiança; na interface real, o tráfego é muito mais diverso e ruidoso, com protocolos como o QUIC e o ICMPv6 a aparecerem espontaneamente sem terem sido pedidos. Foi nesta passagem entre o emulado e o real que os conceitos teóricos das aulas se tornaram concretos.

O sniffer tem limitações conhecidas, das quais as mais relevantes são a impossibilidade de inspecionar tráfego cifrado e o suporte ainda parcial a ICMPv6 (Secção 9). Em trabalho futuro, faria sentido ligar o dashboard web ao sniffer em tempo real através de WebSockets e adicionar suporte a leitura e escrita em formato pcap, permitindo a interoperabilidade com ferramentas como o Wireshark. No estado atual, o sniffer cumpre o que foi pedido e oferece uma base sólida para evoluir.